SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Department of Information Technology

CSA5103 – Cryptography and Network Security

ASSIGNMENT 1 – CO1

Design, Analyse and Validate Classical Encryption Techniques for Securing Sensitive Information

CO1: Apply classical encryption techniques using symmetric and asymmetric ciphers to encrypt and decrypt data for ensuring data security.

Student Name	Register Number	Department / Section	Academic Year / Semester	Faculty Name	Nature of Assignment	Duration	Total Marks	Date of Submission	Date of Evaluation
RAKSHITH R.R	192324156	B-Tech – AI&DS	4th year	DHARANI R	Individual	1 Week	50	10.10.26	11.10.26

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks / Evidence Expected
1	Cipher Design and Selection	5		Select and describe three suitable classical ciphers (Caesar, monoalphabetic substitution, and Vigenère or Hill); justify their use and limitations for the given sensitive message.
2	Encryption and Decryption	5		Show correct key selection, encryption steps, ciphertext, decryption steps and recovery of the original plaintext.
3	Key Analysis and Mathematical Basis	5		Explain key space, modular arithmetic or alphabet mapping used by each selected cipher; calculations must be traceable.
4	Security and Cryptanalysis	5		Analyse vulnerabilities such as brute-force search, frequency analysis, repeated-key patterns and known-plaintext attacks where relevant.
5	Comparison of Cipher Strength	4		Compare the selected methods using key space, implementation effort, likely attacks and suitability for sensitive information.
6	Algorithm / Pseudocode Formulation	4		Provide clear encryption and decryption pseudocode or flowcharts, including input validation and treatment of spaces/punctuation.
7	Worked Example and Verification	4		Use a sample message; verify that decrypt(encrypt(plaintext)) reproduces the original message under the stated rules.
8	Implementation and Testing	4		Implement at least two ciphers in a programming language; include source code, test cases, expected output and actual output.
9	Experimental Evaluation	3		Test multiple messages and key choices; present organised tables of ciphertext, recovered plaintext, execution time or operation counts.
10	Theory-to-Result Discussion	3		Explain how observed results match the cipher design and why classical ciphers are not suitable for modern high-security use.
11	Recommend and Defend a Design	4		Recommend a method for the assignment scenario and defend the decision using correctness, usability and security limitations.
12A	Re-evaluate: Impact Analysis	2		Discuss the impact of message length, key choice and repeated patterns on confidentiality.
12B	Re-evaluate: Attack Evaluation	2		Evaluate likely attacks against the selected ciphers and identify information an attacker could exploit.
12C	Re-evaluate: Adaptation or Redesign	2		Propose a safer modern replacement or layered approach, explaining why classical ciphers are only educational examples.
12D	Re-evaluate: Evidence-Based Defence	2		Defend conclusions using worked examples, test results, key-space reasoning and a comparison table.
	TOTAL	50		Marks to be awarded by the faculty.

OVERALL REMARKS

Faculty feedback:

_____

_____

_____

_____

_____

_____

Student Signature	Faculty Signature	Program Director
		_

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present P = partly present ✗ = missing. Mark the status after checking the student's submitted report and supporting evidence.

Task	Evidence Expected	Status	Note
1	Cipher selection, design rationale and assumptions	☐	Three ciphers described and justified
2	Encryption and decryption working shown	☐	Steps, key and plaintext/ciphertext included
3	Mathematical basis and key-space explanation	☐	Calculations are traceable
4	Security analysis and likely attacks	☐	Relevant weaknesses identified
5	Cipher comparison table	☐	Criteria applied consistently
6	Pseudocode or flowchart	☐	Encryption and decryption steps included
7	Worked example and correctness verification	☐	Recovered plaintext matches original
8	Source code and reproducible tests	☐	Program, test inputs and outputs included
9	Experimental results in tables	☐	Multiple inputs or keys evaluated
10	Theory and experimental results compared	☐	Observations interpreted
11	Recommendation supported by analysis	☐	Trade-offs and limitations stated
12A–D	Impact, attack evaluation, redesign and evidence-based defence	☐	All four parts addressed

IMPLEMENTATION EVIDENCE CHAIN (Tasks 8–9)

Evidence Item	Status	Observation
Source code	☐	Readable source code for each implemented cipher; language and execution instructions stated
Test inputs	☐	Plaintexts, keys and edge cases recorded
Expected behaviour	☐	Expected ciphertext and recovered plaintext documented
Actual output	☐	Program output or screenshots included
Intermediate evidence	☐	Key mapping, modular steps or intermediate transformations shown
Boundary / failure tests	☐	Empty input, spaces, punctuation, invalid keys and repeated characters considered
Interpretation	☐	Results connected to cipher properties and security limitations
Code, data and report consistent	☐	Tables and claims match the submitted program output

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All assignment tasks including 12A–12D answered	☐ Yes ☐ Partly ☐ No
Encryption and decryption shown for each selected cipher	☐ Yes ☐ Partly ☐ No
Keys and assumptions clearly stated	☐ Yes ☐ Partly ☐ No
At least two ciphers implemented and tested	☐ Yes ☐ Partly ☐ No
Same plaintext conventions used consistently	☐ Yes ☐ Partly ☐ No
Source code and reproducible test procedure provided	☐ Yes ☐ Partly ☐ No
Tables / figures / program outputs included	☐ Yes ☐ Partly ☐ No
Cross-references and numbering correct	☐ Yes ☐ Partly ☐ No
No unfilled template placeholders in the student report	☐ Yes ☐ Partly ☐ No
Viva recommended where implementation or calculations need clarification	☐ Yes ☐ No

SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT 1

QUESTIONS

CO1: Apply classical encryption techniques using symmetric and asymmetric ciphers to encrypt and decrypt data for the ensuring data security.

Assignment Title: Evaluate, Design and Defend a Secure Patient-Record Exchange Strategy for a Multi-Hospital Healthcare Network (Total Marks: 50)

Scenario — Confidential Patient Record Exchange

A regional healthcare network links one central hospital with 25 satellite clinics. Clinics send patient records, diagnostic reports, prescriptions and billing data to the central hospital over a public network and receive specialist opinions in return. Personnel hold different clearance levels: Medical Director (Top Secret), Specialist Doctors (Secret), Nurses and Lab Staff (Confidential), Billing and Front-Desk Staff (Restricted), Patients (Public).

The system must:

- Keep diagnoses, especially for sensitive conditions, confidential.
- Detect tampering with prescriptions, dosage and lab values.
- Prevent staff from reading or writing records beyond their clearance.
- Work in rural clinics where staff have limited technical training and hardware is basic.

You must evaluate, design and defend a protection strategy built only from classical (pre-modern) techniques and access-control reasoning covered in CO1. You are expected to judge how strong each technique actually is, not just describe it.

Design Challenge: An insider at a satellite clinic has been copying ciphertexts for months. Hundreds of messages under the same key are now exposed, and many begin with predictable text (e.g., "PATIENT ID", "DIAGNOSIS:", "PRESCRIPTION:"). The network also wants to embed a short report inside medical scan images sent between hospitals. Re-evaluate your design.

- Encrypt clinical notes (200 to 10,000 characters) using Caesar, monoalphabetic, Vigenère and columnar transposition, and derive each key space. Break at least three schemes by frequency analysis, Kasiski or index of coincidence, and rank the schemes by resistance.

- Extend the alphabet to digits and symbols (mod 36) and encrypt IDs and dosages such as PT-20481, AMOX 500MG using a Hill cipher on 3-character blocks. Prove the key's invertibility, compute one inverse, and recover the key by a known-plaintext attack.

- Apply columnar transposition to a lab table of 20+ rows and test whether a changed cell or a swapped row is detected. Design a classical checksum and show where an attacker defeats it.

- Encrypt a 256×256 or larger medical image with two classical methods (intensity shift mod 256 and pixel permutation) and verify decryption. Compare histograms and entropy, and explain which method leaks the image's structure.

- Hide a short report in an image by LSB embedding and in a text circular by a text method, then extract it. Measure PSNR, capacity and chi-square detectability, and test survival under JPEG compression.

- Decide 12+ access requests using Bell-LaPadula with compartments, including declassification and break-glass cases. Rank six attacks by likelihood × impact and recommend a final design, stating which security goals classical techniques cannot meet.

TASK 1 – Evaluate Threats, Attacks and Risks **4 Marks**

- Identify at least six security attacks relevant to the healthcare scenario.
- Classify each as passive or active (eavesdropping, traffic analysis, modification of prescriptions, masquerade as a doctor, replay of old orders, denial of service on the records system).
- Distinguish clearly between threat, vulnerability, attack and risk, with examples from this scenario.
- Rank the attacks by risk using a justified likelihood × impact rating. Consider patient-safety impact, not just privacy.
- Defend your ranking: which attack would you address first, and why?

Evidence Expected: A risk table with reasoned ratings and a justification of the priority order.

TASK 2 – Map and Judge Security Goals **3 Marks**

For each system function (record upload, prescription transmission, specialist consultation), decide which goals matter most: confidentiality integrity availability authentication non-repudiation

- Judge which goals classical ciphers can and cannot deliver.

- Justify why a correctly decrypted message does not prove it came from the right doctor, or that its dosage was not altered.

Evidence Expected: A goal-to-function analysis with an evaluation of the gaps classical techniques leave.

TASK 3 – Apply and Critique the Bell-LaPadula Model **5 Marks**

- Define the security levels and subject/object assignments for the network.
- Apply the simple security property (no read up) and the *-property (no write down) to at least five
- access requests, including two that must be denied. Show the reasoning for each.
- Evaluate the model critically in a medical setting: what happens in an emergency when a nurse needs a Secret-level allergy record? How does the model handle a doctor who must write a note to a lower-level billing file? Discuss tranquility and trusted subjects.
- Judge the model's limitations (no integrity guarantees, covert channels, rigidity in emergencies) and argue whether Bell-LaPadula alone suits healthcare.

Evidence Expected: Worked access decisions, a justified critique, and a clear verdict on the model's suitability.

TASK 4 – Design the Cryptographic Strategies **4 Marks**

Select at least three meaningfully different classical schemes from: Caesar, monoalphabetic substitution, Vigenère (polyalphabetic), Hill cipher, a transposition cipher. For each, provide:

- The encryption and decryption algorithm (pseudocode or formula) and the key definition.
- A worked example on a 20+ character message from the scenario.
- A justification for including it in your comparison.

Evidence Expected: Three distinct schemes with algorithms, worked examples and justified selection.

TASK 5 – Mathematical Analysis of the Schemes **5 Marks**

- Derive the key-space size of each scheme (e.g., 25, 26!, 26^m, number of invertible n×n matrices mod 26, n! permutations).
- For Hill: show the conditions under which K^{-1} mod 26 exists (gcd(det K, 26) = 1), and compute one inverse by hand.
- Express each encryption as a modular function (e.g., C = (aP + b) mod 26 for an affine variant, C = KP mod 26 for Hill).
- Judge whether a large key space guarantees strength. Support your answer with the monoalphabetic example.

Evidence Expected: Correct derivations with a reasoned judgment on key space versus real security.

TASK 6 – Evaluate Cryptanalytic Resistance **4 Marks**

Attack each chosen scheme using a technique suited to it:

- Brute force, frequency analysis, Kasiski examination / index of coincidence, known-plaintext attack on Hill, diagram analysis.
- Explain why each attack succeeds or fails, in terms of letter-frequency preservation, key repetition and linearity.
- Rank the schemes by resistance and justify the ranking with evidence.

Evidence Expected: Worked attacks with reasoning, and a ranking of schemes supported by the results.

TASK 7 – Assess Transposition and Product Ciphers **3 Marks**

- Implement a columnar (or rail-fence) transposition and explain how it diffuses plaintext structure.
- Evaluate whether *substitution followed by transposition* is stronger than either alone. Justify with letter-frequency evidence and an attack attempt.
- Judge whether the added complexity is practical for rural clinic staff doing manual or low-resource encryption.

Evidence Expected: A reasoned evaluation of the combined scheme against its components.

TASK 8 – Evaluate Steganography **3 Marks**

Design a steganographic method (e.g., LSB embedding in a scan image, or text-based hiding) for carrying a short report inside a medical image.

- Compare it with encryption on capacity, imperceptibility, robustness and detectability.
- Evaluate the risk specific to medical imaging: could embedding alter pixel data enough to affect a diagnosis? Judge the trade-off.
- Judge when steganography is appropriate, and argue why hiding is not a substitute for encrypting. Evaluate an encrypt-then-embed approach.

Evidence Expected: A criteria-based comparison with a justified conclusion.

TASK 9 – Implement and Test **4 Marks**

Implement all chosen schemes, the cryptanalysis tools and (where feasible) the steganography method, in a language of your choice.

- Provide correctness tests: decrypt(encrypt(m)) = m, including edge cases (digits in patient IDs, punctuation, non-multiple block length for Hill).
- Use identical plaintexts of 100, 1,000, 10,000 and 100,000 characters across schemes.
- State the testing procedure clearly so it can be reproduced.

Evidence Expected: Working code with a documented, reproducible test procedure.

TASK 10 – Record and Interpret Experimental Evidence **3 Marks**

Present data in tables and graphs for:

- Letter-frequency histograms of plaintext versus each ciphertext.
- Index of coincidence per scheme.
- Encryption/decryption time against message size.
- Attack success rate against ciphertext length.

Evaluate what the data shows. Do not just present it.

Evidence Expected: Clear, comparable results with interpretation tied to Tasks 5 and 6.

TASK 11 – Recommend and Defend the Final Design **4 Marks**

Recommend one scheme, or a layered combination, together with the Bell-LaPadula policy.

- Justify against the alternatives using your mathematical and experimental evidence.
- State the residual risks and limitations honestly. Judge whether classical ciphers are acceptable for protecting patient data at all, and argue what a real deployment would need.
- Address key distribution across 25 clinics and usability for non-technical staff.

Evidence Expected: A defended recommendation grounded in your own analysis, with candid

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design **8 Marks**

Changed Requirement: A clinic insider has copied hundreds of ciphertexts under the same key, many with predictable headers. The network also wants reports hidden inside medical scan images.

You may RETAIN → MODIFY → or REDESIGN your original solution.

A. Impact Analysis (2 Marks)

- What has changed in the threat model (insider versus external attacker)?
- Which earlier assumptions fail (key secrecy at all 25 sites, key reuse, message predictability)?
- Why does a high volume of known-format messages change the risk ranking?

B. Evaluation of the Original Design (2 Marks)

- Test your recommended schemeagainst the new attack capabilities (known-plaintext, repeated-key attacks).
- Support the evaluation with a demonstration or calculation.
- Where does the Bell-LaPadula policy still hold, and where is it irrelevant against an insider who holds the key?

C. Adaptation or Redesign (2 Marks)

- Decide to retain, modify or redesign. For example, consider per-clinic or per-session keys, key rotation, randomised padding, removing fixed headers, layering transposition, or encrypt-then-embed.
- Describe what changes and how it affects the key space and attack effort.

D. Evidence-Based Defence (2 Marks)

Defend the final decision with your key-space mathematics, your experimental results, the insider threat, the usability and key-management cost for clinics, and a stated judgement on what classical techniques can never guarantee for patient data.

Evaluate, Design and Defend a Secure Patient-Record Exchange Strategy for a Multi-Hospital Healthcare Network

Guide to this Report

This report answers the twelve tasks of the assignment in order. Every claim is backed by a calculation, a worked example or a measurement made by the program in the appendices, and every table of results is copied from the program output, so the report, the code and the data agree. The assignment sheet at the front uses mark-sheet numbers; the questions use Task numbers. Table 1 shows how they correspond and where each piece of required evidence is found.

Mark-sheet item (marks)	Answered in	Evidence
1 Cipher design and selection (5)	Task 4, Task 11	Three plus two ciphers described and justified
2 Encryption and decryption (5)	Task 4, Task 9	Keys, step tables, ciphertext, recovered text
3 Key analysis and mathematical basis (5)	Task 5	Key-space derivations, Hill inverse by hand
4 Security and cryptanalysis (5)	Task 6, Task 7	Brute force, frequency, Kasiski, IC, known-plaintext
5 Comparison of cipher strength (4)	Task 6, Task 10	Ranking and comparison tables
6 Algorithm / pseudocode (4)	Task 4	Pseudocode, flowchart with validation
7 Worked example and verification (4)	Task 4, Task 9	decrypt(encrypt(m)) = m shown and tested
8 Implementation and testing (4)	Task 9, Appendices	Source code, 40 tests, expected vs actual
9 Experimental evaluation (3)	Task 10	Tables and graphs for four message sizes
10 Theory-to-result discussion (3)	Task 10	Interpretation tied to Tasks 5 and 6

11 Recommend and defend a design (4)	Task 11	Layered design, trade-offs, key distribution
12A–12D Re-evaluate (8)	Task 12 A, B, C, D	Impact, attack demonstration, redesign, defence

Table 1: Mark-sheet items and the sections that answer them

Implementation evidence (Tasks 8–9 chain)	**Where it is in this report**
Source code, language, run instructions	Appendices A–F (code); instructions in this section
Test inputs	Task 9: edge-case, size and key tables
Expected behaviour	Task 9: expected-versus-actual table (40 tests)
Actual output	Appendix G (program output) and the tables of Tasks 6, 7, 8, 10 and 12
Intermediate evidence	Task 4: step tables for every cipher; Task 5: Hill inverse
Boundary / failure tests	Task 9: empty input, spaces, punctuation, digits, invalid keys, repeated letters
Interpretation	Tasks 6, 7, 10
Code, data and report consistent	Every number comes from run_all.py, test_all.py and worked.py (seed 2026)

Table 2: Implementation evidence chain

Conventions and assumptions used throughout

- **Plaintext convention.** Letter ciphers (Caesar, monoalphabetic, Vigenère, columnar transposition) use the 26 letters A–Z: text is converted to upper case and spaces, digits and punctuation are removed (normalised). decrypt(encrypt(m)) therefore equals the normalised m. Hill and the 36-symbol Vigenère variant use A–Z then 0–9 (A = 0 … Z = 25, 0 = 26 … 9 = 35); other characters are removed and short blocks are padded with X, the pad count being kept so it can be removed.
- **Attacker model.** The attacker knows the algorithm (Kerckhoffs' principle), the clinical vocabulary and public English letter statistics; n-gram statistics are learnt from a separate synthetic clinical corpus (different random seed from every test message). This is a deliberately strong attacker, which is the correct assumption for a defender.
- **Data.** No real patient data is used. Clinical notes, lab tables and the 256×256 scan are synthetic. The generated notes use a limited vocabulary, so their letter statistics are slightly more regular than natural English (index of

coincidence about 0.072 against 0.067 for English); results therefore show the order of magnitude, not exact thresholds for real notes.

- **Reproduction.** Install Python 3 with numpy, scipy, pillow and matplotlib; keep all .py files in one folder; run python test_all.py (40 tests, prints expected and actual), python run_all.py (all experiments, about two minutes, writes results.json), python worked.py and python figs_co1.py. All random choices use fixed seeds.

Task 1 – Evaluate Threats, Attacks and Risks

Definitions with examples from this scenario

Term	Meaning	Example in the healthcare network
Threat	A potential cause of harm: an actor or event that could exploit a weakness.	A dishonest employee at a satellite clinic who can copy network traffic.
Vulnerability	A weakness that a threat can use.	One shared key at all 25 clinics; no integrity check on prescriptions; messages that start with a fixed header.
Attack	An action that uses a vulnerability to realise a threat.	The insider collects ciphertexts, guesses the header, recovers the key and changes a dosage.
Risk	The expected harm: likelihood × impact for a given attack.	Altering a prescription has likelihood 4 and impact 5, so risk 20 of 25.

Table 3: Threat, vulnerability, attack and risk

Attacks, classification and rating

Likelihood is scored 1 (rare) to 5 (almost certain) for the current design on a public network. Impact is scored 1–5 with patient safety weighted above privacy: 1 negligible; 2 minor privacy leak; 3 serious privacy breach of individual patients; 4 delayed care or a large breach; 5 risk to life or bulk exposure of the whole network. Risk = likelihood × impact; ties are broken by impact (patient safety first).

Rank	Attack	Type	L	I	L×I	Justification
1	A3 Modification of prescription / dosage	active	4	5	**20**	No integrity protection: an altered dose is accepted as valid; wrong dose can kill.

2	A5 Replay of old orders	active	4	4	**16**	A captured valid order can be sent again (duplicate dose, repeated test, cancelled order revived).
3	A4 Masquerade as a doctor	active	3	5	**15**	Shared keys give no proof of origin; a forged order or record request is accepted.
4	A1 Eavesdropping on the public network	passive	5	3	**15**	Anyone on the path can copy traffic; harm is privacy (diagnoses), not direct physical harm.
5	A6 Denial of service on the records system	active	3	4	**12**	Care is delayed, but data is not changed or disclosed.
6	A2 Traffic analysis	passive	4	2	**8**	Message sizes and timing reveal which clinic sends which kind of record, but not the content.

Table 4: Risk register: six attacks ranked by likelihood × impact

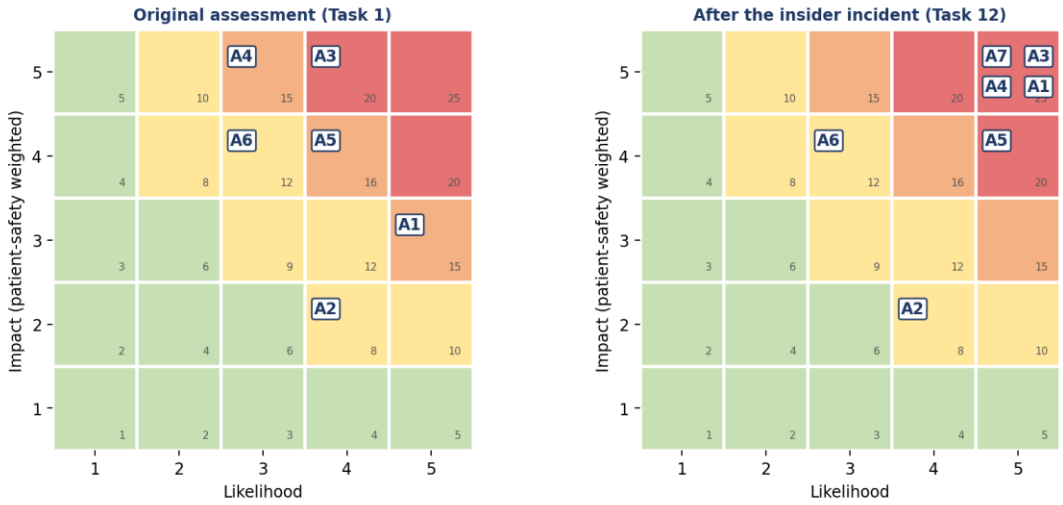


Figure 1: Risk matrix: left, original assessment (this task); right, assessment after the insider incident (Task 12A)

Defence of the ranking

The first attack to address is A3, modification of prescriptions and dosages. It has the highest score (4 × 5 = 20) and its consequences are irreversible: a changed dose can kill a patient, whereas eavesdropping (A1) harms

privacy. Replay (A5, 16) comes second because a captured valid order can be repeated by anyone who saw it. Masquerade as a doctor (A4) and eavesdropping (A1) share a score of 15, but A4 ranks higher because its impact is 5 (patient safety). Denial of service (A6, 12) delays care but does not corrupt it, and traffic analysis (A2, 8) leaks only metadata. A purely confidentiality-driven ranking would have put eavesdropping first; the patient-safety weighting changes that, and it also shows the central difficulty of this assignment: the top-ranked attacks (A3, A5, A4) are integrity and authentication attacks, which classical ciphers do not address at all (Task 2).

Answer to Task 1

Six attacks were classified (passive: eavesdropping, traffic analysis; active: modification, masquerade, replay, denial of service) and ranked by a justified likelihood × impact score with patient safety weighted: A3 modification (20), A5 replay (16), A4 masquerade (15), A1 eavesdropping (15), A6 denial of service (12), A2 traffic analysis (8). Modification of prescriptions is addressed first because its harm is irreversible and no classical cipher prevents it.

Task 2 – Map and Judge Security Goals

The importance of each goal was rated for each function (H high, M medium, L low) from the harm that its failure causes.

System function	Confidentiality	Integrity	Availability	Authentication	Non-repudiation
Record upload (clinic to hospital)	**H**	**H**	M	**H**	M
Prescription transmission	M	**H**	**H**	**H**	**H**
Specialist consultation	**H**	**H**	M	**H**	M

Table 5: Goals that matter most for each function (H = high priority)

Prescriptions need integrity, authentication and non-repudiation above all: a wrong or forged dose is the most dangerous failure, and a doctor must not be able to deny, or be blamed for, an order. Record uploads and consultations need confidentiality, but also integrity (a corrupted allergy entry is dangerous) and authentication of the sender.

Which goals can classical ciphers deliver?

Goal	Delivered?	Judgement

Confidentiality	**Partly**	Only against a casual observer and only for short messages: all schemes here were broken in the experiments of Tasks 6–7 with a few hundred letters, and the insider attack of Task 12 breaks them with a handful of messages.
Integrity	**No**	Every ciphertext decrypts to something and the cipher adds no redundancy, so changes go unnoticed (demonstrated below and in Task 7).
Availability	**No**	Encryption does not stop denial of service; it only adds processing work.
Authentication	**No**	The key is shared; anyone holding it can produce a valid-looking message from any clinic.
Non-repudiation	**No**	With a shared key both sides can create the same ciphertext, so no one can prove who wrote a message.

Table 6: What classical ciphers can and cannot deliver

Why a correct decryption proves nothing about origin or content

Every shift, substitution, Vigenère and transposition ciphertext of the right alphabet decrypts to some plaintext; there is no internal check that tells the receiver the result is the one the sender wrote. Three consequences follow. (1) Origin: all 25 clinics share keys, so a message that decrypts into a sensible text shows only that its author knew the key, not that Dr X wrote it. (2) Content: the additive ciphers are malleable. A demonstration with the 36-symbol Vigenère cipher (key KEY7) on the prescription AMOX500MG:

Step	Text	Explanation
Original plaintext	AMOX500MG	Amoxicillin 500 mg
Sent ciphertext	KQCUF4OJQ	Vigenère-36, key KEY7
Attacker adds +4 to the 5th symbol	KQCUJ4OJQ	He does not know the key; he only knows that the fifth symbol is the first digit
Receiver decrypts the forged text	**AMOX900MG**	Dose changed from 500 mg to 900 mg; decryption succeeds with no warning

Table 7: Undetected dosage change by adding a constant to one ciphertext symbol

(3) Replay: an old but valid ciphertext is accepted again because nothing in it states when or for whom it was written. These gaps must be closed by other means: integrity codes, signatures, sequence numbers and access control.

Answer to Task 2

Prescription transmission needs integrity, authentication, availability and non-repudiation most; record upload and consultation need confidentiality, integrity and authentication. Classical ciphers deliver partial confidentiality only. They give no integrity, authentication, non-repudiation or availability, and a correct decryption proves neither the sender nor the dosage, as the AMOX500MG → AMOX900MG forgery shows.

Task 3 – Apply and Critique the Bell-LaPadula Model

Levels, subjects and objects

Levels, from lowest to highest: PUBLIC < RESTRICTED < CONFIDENTIAL < SECRET < TOP SECRET. Because clearance level alone is too coarse for diagnoses such as HIV status, compartments are added: GEN (general), ONC (oncology), PSY (psychiatry), HIV. A label is (level, compartments); label A dominates label B when A's level is at least B's and A's compartments include all of B's.

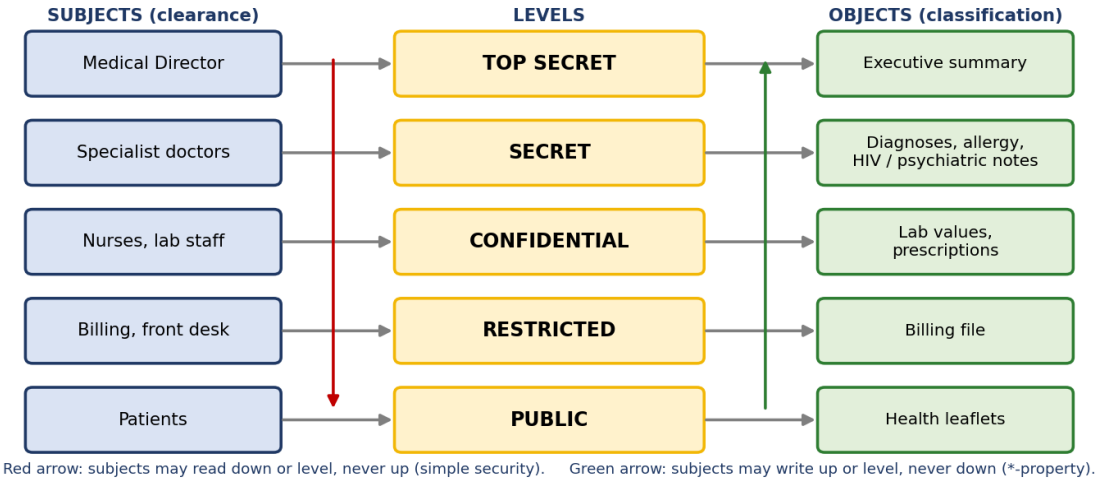


Figure 2: Bell-LaPadula structure of the network: subjects, levels and objects

Subject	Level	Compartments	Role
Medical Director	TOP SECRET	GEN, ONC, PSY, HIV	Trusted subject (may declassify)
Dr Rao (oncologist)	SECRET	GEN, ONC	Specialist doctor
Dr Iyer (psychiatrist)	SECRET	GEN, PSY	Specialist doctor
Nurse Meena, Lab tech Arun	CONFIDENTIAL	GEN	Nurses and lab staff

Billing clerk	RESTRICTED	none	Billing and front desk
Patient	PUBLIC	none	Patient

Table 8: Subjects and clearances

Object	Level	Compartments	Content
Executive summary	TOP SECRET	all four	Network-wide clinical overview
Oncology diagnosis / psychiatric notes / HIV result	SECRET	ONC / PSY / HIV	Sensitive diagnoses
Allergy record	SECRET	GEN	Needed urgently in emergencies
Lab values, prescriptions	CONFIDENTIAL	GEN	Clinical data
Billing file, declassified lab summary	RESTRICTED	none	Administrative data
Health leaflet	PUBLIC	none	Public information

Table 9: Objects and classifications

Rules applied to access requests

Simple security property: a subject may read an object only if the subject's label dominates the object's (no read up). *-property: a subject may write an object only if the object's label dominates the subject's (no write down), so that high-level information cannot be copied into a lower-level object. Trusted subjects are exempt from the *-property for declassification, and a break-glass override is outside the model and must be logged. The program blp.py evaluated 16 requests:

#	Subject	Action	Object	Decision	Reasoning
1	Dr Rao (oncologist)	read	oncology diagnosis	**ALLOW**	Clearance dominates the object's label
2	Nurse Meena	read	oncology diagnosis	**DENY**	Read up: clearance level too low
3	Nurse Meena	read	lab values	**ALLOW**	Clearance dominates the object's label
4	Billing clerk	read	prescription	**DENY**	Read up: clearance level too low
5	Dr Iyer (psychiatrist)	read	oncology diagnosis	**DENY**	Level is enough but compartment ['ONC'] is missing (same level, different compartment)

6	Dr Rao (oncologist)	read	HIV test result	**DENY**	Level is enough but compartment ['HIV'] is missing (compartment not held)
7	Medical Director	read	psychiatric notes	**ALLOW**	Clearance dominates the object's label
8	Lab tech Arun	write	lab values	**ALLOW**	Write up or equal is allowed
9	Lab tech Arun	write	billing file	**DENY**	Write down: would copy higher-level data into a lower object (write down)
10	Billing clerk	write	lab values	**ALLOW**	Write up or equal is allowed (write up: allowed by BLP, but harmful)
11	Dr Rao (oncologist)	write	billing file	**DENY**	Write down: would copy higher-level data into a lower object (doctor must record a billing note)
12	Medical Director	write	lab summary (declassified)	**ALLOW (trusted)**	Trusted subject writes a sanitised summary down; logged (declassification by trusted subject)
13	Nurse Meena	read	allergy record	**ALLOW (override)**	BLP denies (read up); emergency override with audit and review (emergency: Secret allergy record)
14	Patient	read	health leaflet	**ALLOW**	Clearance dominates the object's label
15	Patient	read	prescription	**DENY**	Read up: clearance level too low (patient's own record)
16	Dr Rao (oncologist)	write	executive summary	**ALLOW**	Write up or equal is allowed (write up)

Table 10: Sixteen access decisions (program output of blp.py)

Requests 2, 4, 5, 6, 9, 11 and 15 are denied (seven denials, exceeding the required two). Requests 5 and 6 show why compartments matter: a psychiatrist and an oncologist have the same level as the data, yet are denied because they lack the compartment.

Critical evaluation in a medical setting

- **Emergency (request 13).** A nurse needs the Secret allergy record of an unconscious patient. Bell-LaPadula denies the read; delay can kill. A break-glass procedure grants a time-limited read, records who, when and why, and triggers a review. This is not part of the model and it temporarily violates tranquility.
- **Doctor writing to a lower-level file (requests 11 and 12).** The *-property forbids Dr Rao from writing a billing note, because the note might carry Secret information. The usual remedies are a trusted subject (the Medical Director, or a guard process) that downgrades a sanitised summary after review, or the doctor logging in at a lower level for that task. The second remedy changes the subject's level during a session, which weak tranquility allows if no information is lost and strong tranquility forbids.
- **Tranquility and trusted subjects.** Strong tranquility (labels never change while the system runs) is too rigid for care, where a patient's data may be reclassified or declassified; weak tranquility allows changes that do not break the policy. Trusted subjects are powerful and are exactly the weak point an attacker would aim for; they must be few, audited and given dual control.
- **No integrity (request 10).** The model allows the billing clerk to write up into the lab values, because writing up leaks nothing. It does not stop a low-level user from corrupting high-level data, so a wrong lab value can be entered. Integrity needs a separate model (Biba or Clark-Wilson) or dual verification.
- **Covert channels and own-record access.** A low-level user can learn that a Secret record exists from different error messages or response times. The model also cannot express that a patient may read his own prescription (request 15 is denied) because it has no concept of ownership.

Verdict on Task 3

Bell-LaPadula with compartments is a sound way to state who may read and write which record, and it blocks read-up and write-down errors, as the 16 worked decisions show. It is not sufficient for healthcare on its own: it provides no integrity, cannot express emergencies, ownership or need-to-know beyond compartments, and depends on trusted subjects and break-glass procedures that sit outside the model. It should be combined with an integrity policy, audit logging and cryptographic protection.

Task 4 – Design the Cryptographic Strategies

Five classical schemes were implemented. Caesar, monoalphabetic substitution and Vigenère are the three required meaningfully different schemes (shift, arbitrary substitution, polyalphabetic substitution); columnar transposition and the Hill cipher are added because the assignment's design challenge needs a permutation cipher and a linear block cipher. Worked examples use the 26-letter scenario message "SEND LAB REPORT FOR PATIENT NOW" (normalised: SENDLABREPORTFORPATIENTNOW). All encryptions below are verified by decrypting back to the original.

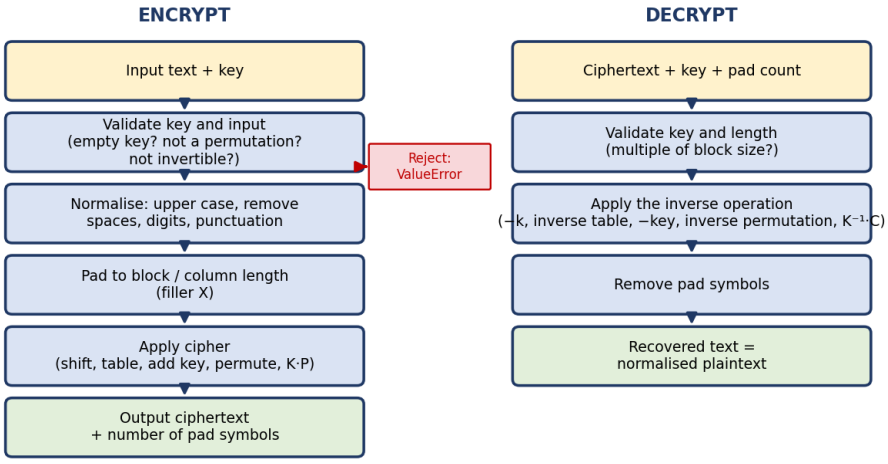


Figure 3: Common flowchart for encryption and decryption, including input validation and handling of spaces and punctuation

4.1 Caesar cipher

Item	Definition
Key	Integer k, $0 \le k \le 25$ (k = 11 in the example); any non-integer is rejected.
Encryption	$C_i = (P_i + k) \bmod 26$
Decryption	$P_i = (C_i - k) \bmod 26$
Pseudocode	validate k is integer; for each char c of normalise(text): output A[(index(c)+k) mod 26] (decrypt: use −k)

Table 11: Caesar cipher definition

Position	Plain letter	Value	(P + 11) mod 26	Cipher value	Cipher letter

1	S	18	18 + 11 = 29 − 26 = 3	3	**D**
2	E	4	4 + 11 = 15	15	**P**
3	N	13	13 + 11 = 24	24	**Y**
4	D	3	3 + 11 = 14	14	**O**
5	L	11	11 + 11 = 22	22	**W**
6	A	0	0 + 11 = 11	11	**L**
7	B	1	1 + 11 = 12	12	**M**
8	R	17	17 + 11 = 28 − 26 = 2	2	**C**
9	E	4	4 + 11 = 15	15	**P**
10	P	15	15 + 11 = 26 − 26 = 0	0	**A**
11	O	14	14 + 11 = 25	25	**Z**
12	R	17	17 + 11 = 28 − 26 = 2	2	**C**

Table 12: Caesar worked example (first 12 letters of the message shown)

Result: SENDLABREPORTFORPATIENTNOW → **DPYOWLMCPAZCEQZCALETPYEYZH** → decrypts to **SENDLABREPORTFORPATIENTNOW**. Justification: the simplest shift cipher, the baseline for the comparison; it shows what a tiny key space does.

4.2 Monoalphabetic substitution

Item	Definition
Key	A permutation of the 26 letters (26! keys). The example uses the keyword alphabet from HEALTHCARE: the keyword without repeated letters followed by the unused letters in order.
Encryption	$C_i = \pi(P_i)$ (table look-up)
Decryption	$P_i = \pi^{-1}(C_i)$ (inverse table)
Pseudocode	validate key has 26 distinct letters; build table A→key for each char c of normalise(text): output table[c]

Table 13: Monoalphabetic substitution definition

Row	Alphabet
Plain	ABCDEFGHIJKLMNOPQRSTUVWXYZ
Cipher	**HEALTCRBDFGIJKMNOPQSUVWXYZ**

Table 14: Key mapping of the example

Result: SENDLABREPORTFORPATIENTNOW → **QTKLIHEPTNMPSCMPNHSDTKSKMW** → decrypts to **SENDLABREPORTFORPATIENTNOW**. Justification: the huge key space (26!) looks strong; it is included to test whether key space alone gives security (Task 5).

4.3 Vigenère cipher

Item	Definition
Key	A keyword of m letters (26^m keys). Example key MEDICAL (m = 7).
Encryption	C_i = (P_i + K_(i mod m)) mod 26
Decryption	P_i = (C_i − K_(i mod m)) mod 26
Pseudocode	validate key has letters; j = 0 for each char c of normalise(text): output A[(idx(c)+idx(key[j mod m])) mod 26]; j = j + 1

Table 15: Vigenère cipher definition

i	P	p	K	k	(p + k) mod 26	C
1	S	18	M	12	18 + 12 = 30 − 26 = 4	**E**
2	E	4	E	4	4 + 4 = 8	**I**
3	N	13	D	3	13 + 3 = 16	**Q**
4	D	3	I	8	3 + 8 = 11	**L**
5	L	11	C	2	11 + 2 = 13	**N**
6	A	0	A	0	0 + 0 = 0	**A**
7	B	1	L	11	1 + 11 = 12	**M**
8	R	17	M	12	17 + 12 = 29 − 26 = 3	**D**
9	E	4	E	4	4 + 4 = 8	**I**
10	P	15	D	3	15 + 3 = 18	**S**
11	O	14	I	8	14 + 8 = 22	**W**
12	R	17	C	2	17 + 2 = 19	**T**
13	T	19	A	0	19 + 0 = 19	**T**
14	F	5	L	11	5 + 11 = 16	**Q**

15	O	14	M	12	14 + 12 = 26 − 26 = 0	**A**
16	R	17	E	4	17 + 4 = 21	**V**
17	P	15	D	3	15 + 3 = 18	**S**
18	A	0	I	8	0 + 8 = 8	**I**
19	T	19	C	2	19 + 2 = 21	**V**
20	I	8	A	0	8 + 0 = 8	**I**
21	E	4	L	11	4 + 11 = 15	**P**
22	N	13	M	12	13 + 12 = 25	**Z**
23	T	19	E	4	19 + 4 = 23	**X**
24	N	13	D	3	13 + 3 = 16	**Q**
25	O	14	I	8	14 + 8 = 22	**W**
26	W	22	C	2	22 + 2 = 24	**Y**

Table 16: Vigenère worked example, all 26 letters, key MEDICAL

Result: SENDLABREPORTFORPATIENTNOW → **EIQLNAMDISWTTQAVSIVIPZXQWY** → decrypts to **SENDLABREPORTFORPATIENTNOW**. Justification: it hides letter frequencies by using seven different shifts, so it is the classical answer to frequency analysis and the natural candidate for a recommendation.

4.4 Columnar transposition

Item	Definition
Key	A permutation of k column numbers (k! keys); example k = 7, read order of columns 5, 3, 7, 1, 4, 6, 2.
Encryption	Write the padded text row by row in k columns; read the columns in key order.
Decryption	Split the ciphertext into k equal blocks, place block j in column perm[j], read row by row, remove padding.
Pseudocode	pad text with X to a multiple of k; cipher = join(text[c::k] for c in perm) decrypt: rows = n/k; col[perm[j]] = cipher[j*rows:(j+1)*rows]; read rows

Table 17: Columnar transposition definition

Row	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6	Col 7
1	S	E	N	D	L	A	B
2	R	E	P	O	R	T	F

3	O	R	P	A	T	I	E
4	N	T	N	O	W	X	X
Read order	**4**	**7**	**2**	**5**	**1**	**6**	**3**

Table 18: Columnar transposition grid (the last two cells are the filler XX); the bottom row shows when each column is read

Result: SENDLABREPORTFORPATIENTNOWXX → **LRTWNPPNBFEXSRONDOAOATIXEERT** → decrypts to **SENDLABREPORTFORPATIENTNOW**. Justification: it changes positions, not letters, which gives a different weakness profile (letter frequencies unchanged) and allows the product cipher of Task 7.

4.5 Hill cipher on 3-symbol blocks (mod 36)

Item	**Definition**
Key	A 3×3 matrix K over Z36 with gcd(det K, 36) = 1. Example K = [[1,1,3],[1,2,1],[2,8,1]], det K = 7.
Encryption	$C = K \cdot P \pmod{36}$ for each block P of 3 symbols (column vector)
Decryption	$P = K^{-1} \cdot C \pmod{36}$, $K^{-1} = \det(K)^{-1} \cdot \text{adj}(K) \pmod{36}$
Pseudocode	validate gcd(det K, 36) = 1; normalise to A-Z0-9; pad with X to a multiple of 3 for each block: output (K · block) mod 36; remember number of pad symbols

Table 19: Hill cipher definition (alphabet A–Z = 0–25, 0–9 = 26–35)

Block	**Vector P**	**K·P (no reduction)**	**mod 36**	**Cipher**	**Check**
PT2	(15, 19, 28)	(118, 81, 210)	(10, 9, 30)	**KJ4**	ok
048	(26, 30, 34)	(158, 120, 326)	(14, 12, 2)	**OMC**	ok
1XX	(27, 23, 23)	(119, 96, 261)	(11, 24, 9)	**LYJ**	ok

Table 20: Hill worked example: patient ID "PT-20481" is normalised to PT20481, padded with 2 X

Result: PT-20481 → **KJ4OMCLYJ** → decrypts to **PT20481**. The dosage "AMOX 500MG" gives SCCYD6UUK and decrypts to AMOX500MG. Justification: Hill is the only scheme here that mixes several symbols at once (diffusion), and mod 36 lets it carry digits in IDs and dosages.

Answer to Task 4

Five schemes were defined with key, formula, pseudocode, flowchart and a worked example on a 26-letter scenario message. Caesar (DPYOWLMCPAZCEQZCALETPYEYZH), monoalphabetic (QTKLIHEPTNMPSCMPNHSDTKSKMW), Vigenère (EIQLNAMDISWTTQAVSIVIPZXQWY) and columnar transposition (LRTWNPPNBFEXSRONDOAOATIXEERT) all decrypt back to SENDLABREPORTFORPATIENTNOW; Hill encrypts PT-20481 as KJ4OMCLYJ and recovers PT20481. Selection is justified by covering the three classical mechanisms: shift or substitution, polyalphabetic key repetition, and permutation, plus a linear block cipher.

Task 5 – Mathematical Analysis of the Schemes

Key spaces

Scheme	Key and counting argument	Number of keys	Bits (log2)
Caesar	shift k, $1 \leq k \leq 25$ ($k = 0$ is the identity)	25	4.6
Monoalphabetic	any permutation of 26 letters: 26!	403,291,461,126,605,635,584,000,000	88.4
Vigenère, m = 7	26^7 (each key letter is free)	8,031,810,176	32.9
Vigenère, m = 12	26^12	95,428,956,661,682,176	56.4
Columnar, 7 columns	7! orders of the columns	5,040	12.3
Columnar, 10 columns	10!	3,628,800	21.8
Columnar, 18 columns	18! (width of the lab table)	6,402,373,705,728,000	52.5
Hill 3×3 mod 26	invertible matrices (formula below)	1,634,038,189,056	40.6
Hill 3×3 mod 36	invertible matrices	19,016,370,487,296	44.1
Monoalphabetic + columnar (7)	product of the two key spaces	≈ 10^30.3	100.7

Table 21: Key-space derivation for each scheme

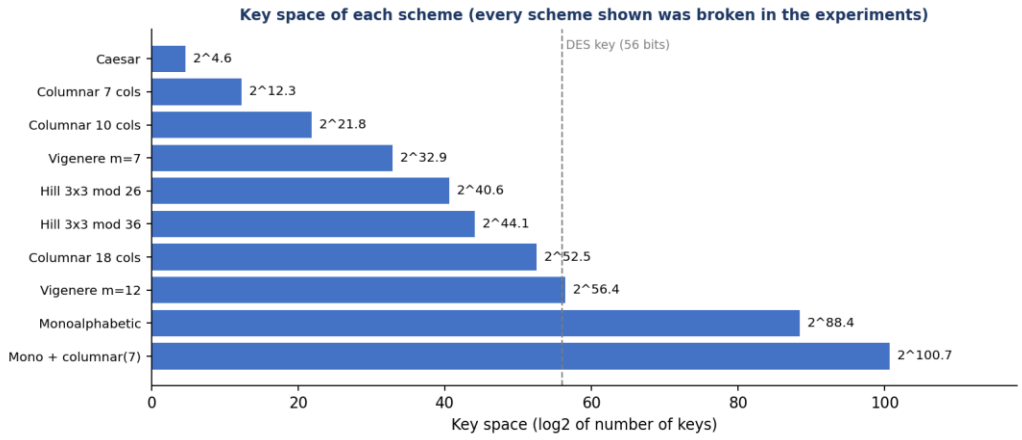


Figure 4: Key space in bits; every scheme in the chart was nevertheless broken in the experiments of Task 6

Modular-arithmetic form of each cipher

Scheme	Encryption / decryption	Condition on the key
Caesar	C = (P + k) mod 26; P = (C − k) mod 26	any integer k; every k is invertible
Affine (variant)	C = (a·P + b) mod 26; P = a^{-1}(C − b) mod 26	gcd(a, 26) = 1 so only φ(26) = 12 values of a; 12 × 26 = 312 keys
Vigenère	C_i = (P_i + K_(i mod m)) mod 26	any key letters; period m
Hill (n = 3)	C = K·P (mod 36); P = K^{-1}·C (mod 36)	K^{-1} exists iff gcd(det K, 36) = 1
Mono / columnar	permutations, not arithmetic: C_i = π(P_i) or C_(σ(i)) = P_i	π and σ must be permutations

Table 22: Modular expressions

Hill cipher: when is the key valid, and one inverse by hand

K is invertible modulo m exactly when det K has a multiplicative inverse modulo m, that is gcd(det K, m) = 1. Since 36 = 4 × 9, det K must be odd and not divisible by 3. For K = [[1,1,3],[1,2,1],[2,8,1]] the determinant is 1(2·1 − 1·8) − 1(1·1 − 1·2) + 3(1·8 − 2·2) = −6 + 1 + 12 = 7, and gcd(7, 36) = 1, so K is a valid key.

Step	Calculation
1 inverse of det	7 × 31 = 217 = 6 × 36 + 1, so $7^{-1} \equiv 31$ (mod 36)
2 adjugate adj(K)	transpose of the cofactors, reduced mod 36 = [[30,23,31],[1,31,2],[4,30,1]]

3 inverse	$K^{-1} = 31 \cdot \text{adj}(K) \bmod 36 = [[30,29,25],[31,25,26],[16,30,31]]$
4 check	$K \cdot K^{-1} \bmod 36 = [[1,0,0],[0,1,0],[0,0,1]]$ (identity)

Table 23: Computing K^{-1} mod 36 by hand and checking it by program

A matrix can be valid for one alphabet and invalid for another. The textbook key [[6,24,1],[13,16,10],[20,17,15]] has determinant 441: modulo 26 it is 25 with gcd 1 (valid), but modulo 36 it is 9 with gcd 9 (not invertible). The program rejects it with a ValueError instead of producing ciphertext that could never be decrypted.

Counting valid keys: the number of invertible n×n matrices modulo a prime power p^e is p^((e−1)n²) · ∏(p^n − p^i). For n = 3: modulo 4 there are $2^9 \cdot 7 \cdot 6 \cdot 4 = 86{,}016$; modulo 9 there are $3^9 \cdot 26 \cdot 24 \cdot 18 = 221{,}079{,}456$; by the Chinese remainder theorem the total modulo 36 is their product 19,016,370,487,296, which agrees with the program (19,016,370,487,296). Only 18.7 % of all 3×3 matrices modulo 36 are invertible (30.1 % modulo 26), so a random key must always be tested.

Does a large key space guarantee security?

No. The monoalphabetic key space is 2^88.4, far larger than DES's 2^56, yet the program in Task 6 recovers the key from 448 letters with ≥ 90 % success in under a second. The reason is that the attack never searches the key space: letter frequencies and trigram statistics let it improve a guess one swap at a time, so it evaluates a few thousand keys, not 10^26. The same holds for Vigenère (a key space of 2^32.9 is split into seven independent Caesar problems of 26 keys each) and for columnar transposition (the column order is found by joining columns whose edges form English letter pairs). A large key space is necessary, never sufficient: strength also needs diffusion (a change spreads) and confusion (the key is not visible in the statistics), which these ciphers lack.

Answer to Task 5

Key spaces: Caesar 25 (2^4.6); monoalphabetic 26! ≈ 4.03×10^26 (2^88.4); Vigenère 26^m (m = 7: 2^32.9; m = 12: 2^56.4); columnar k! (7: 2^12.3; 18: 2^52.5); Hill 3×3 invertible matrices 1.63×10^12 mod 26 (2^40.6) and 1.90×10^13 mod 36 (2^44.1). Hill needs gcd(det K, 36) = 1; for K above det = 7, $7^{-1} = 31$ and K^{-1} was computed and verified. Large key spaces do not guarantee security: a 2^88 monoalphabetic key falls to frequency and n-gram analysis with a few hundred letters.

Task 6 – Evaluate Cryptanalytic Resistance

Each scheme was attacked with a technique suited to it. All attacks were run by program (attacks.py) against messages the attacker had not seen; success means at least 90 % of the plaintext letters recovered.

Scheme	Attack	What it exploits	Work

Caesar	Brute force over 25 shifts, chi-square against English frequencies	Tiny key space; shifting keeps the letter-frequency pattern	25 trials
Monoalphabetic	Frequency-rank guess, then hill climbing scored by trigram statistics	Letter and letter-group frequencies are unchanged by substitution	≈ 10^4–10^5 key tests
Vigenère	Kasiski examination and index of coincidence for the key length, then frequency analysis per column	Key repeats with period m: repeated plaintext groups give ciphertext repeats at distances that are multiples of m	7 × 26 trials
Columnar	Column-adjacency bigram scores and dynamic programming (Held–Karp) over column orders	Letters are unchanged; neighbouring columns must form English letter pairs	k^2 scores
Hill	Known-plaintext: solve $K = C \cdot P^{-1}$ from 3 independent blocks	$C = K \cdot P$ is linear	one matrix inverse

Table 24: Attack used against each scheme

Worked break of a 1,008-letter note

Scheme	Result	Evidence
Caesar	key k = 11 found, 100 % correct	lowest chi-square of the 26 trials
Monoalphabetic	100 % of letters correct	key recovered; true cipher alphabet XYLKWBFZTNJRQAHVGMUOPDICSE
Vigenère	key MEDICAL recovered exactly (m = 7), 100 % correct	Kasiski: 146 repeated-trigram distances; divisors most often 7 (118 votes), then 2, 14, 3, 21
Columnar	7 columns found, order recovered, 100 % correct	highest bigram likelihood

Table 25: Single worked break per scheme (1,008 letters)

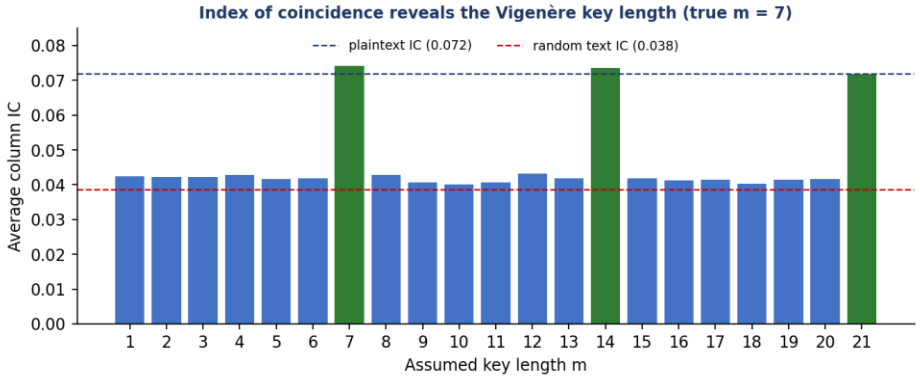


Figure 5: Index of coincidence: columns of the Vigenère ciphertext look like English only when m is a multiple of the true key length 7

The Kasiski and IC results agree. In English each column of a correctly guessed key length is a Caesar cipher with index of coincidence close to the plaintext value (0.072 here); for a wrong length the columns mix several shifts and the value falls towards random text (0.038).

Hill cipher: known-plaintext attack

Every note starts with the predictable header PATIENT ID … DIAGNOSIS, normalised to PATIENTIDPTDIAGNOSIS. Six plaintext blocks and their ciphertext blocks were therefore known (PAT→A8N, IEN→P3Z, TID→AC7, PTD→HUF, IAG→0OW, NOS→JXM). Blocks 1, 2 and 4 (PAT, IEN, PTD) form an invertible plaintext matrix modulo 36 (blocks 1–3 do not), so $K = C \cdot P^{-1}$ gave [[1,1,3],[1,2,1],[2,8,1]], equal to the true key (match = true). With random known blocks, three blocks are invertible with probability 19.5 % (theory 18.7 %), and on average 4.9 known blocks (about 15 letters) suffice. The Hill cipher's flat letter histogram (IC 0.0288, Task 10) protects against single-letter frequency analysis, but its linearity makes it collapse under a handful of known symbols.

Success against ciphertext length

Length	Caesar	Mono (freq. only)	Mono (n-gram)	Vigenère m=7	Columnar k=7	Mono + columnar
56	100 %	0 %	30 %	0 %	86 %	0 %
112	100 %	0 %	57 %	33 %	99 %	8 %
224	100 %	0 %	87 %	70 %	100 %	25 %
448	100 %	0 %	97 %	100 %	100 %	67 %
896	100 %	0 %	100 %	100 %	100 %	83 %

1792	100 %	0 %	100 %	100 %	100 %	100 %
3584	100 %	10 %	100 %	100 %	100 %	100 %
7168	100 %	7 %	100 %	100 %	100 %	100 %

Table 26: Attack success rate (≥ 90 % of letters recovered). Trials per cell: Caesar, Vigenère, columnar 100; monoalphabetic 30; product 12

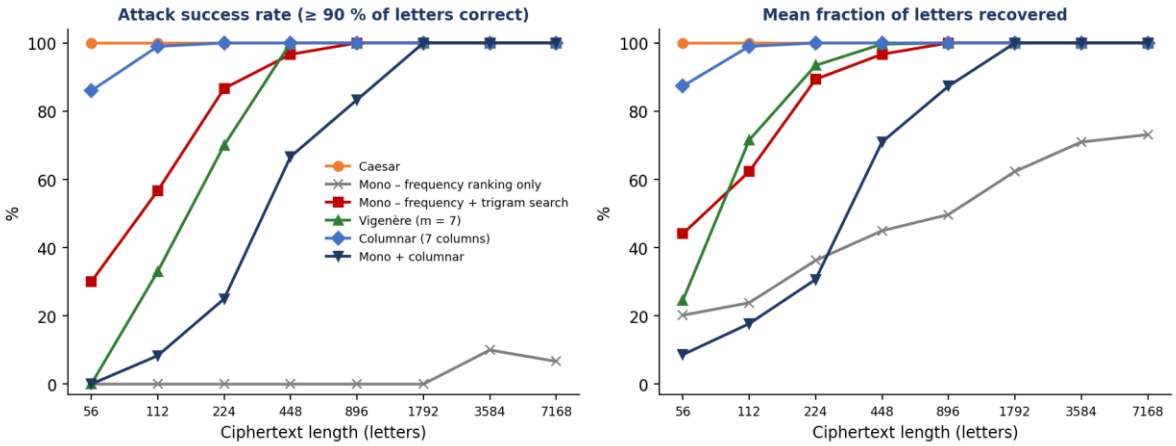


Figure 6: Attack success versus ciphertext length

Ranking by resistance

Rank	Scheme	Letters needed for ≥ 90 % success	Reason
1	**Caesar (weakest)**	56 or fewer	25 keys; the whole frequency pattern is just shifted.
2	**Columnar transposition alone**	112	Letters unchanged (IC stays 0.072); only 7! orders; rows of neighbouring columns are easy to test.
3	**Monoalphabetic**	448	2^88 keys, but frequency and trigram structure survive; at 56–112 letters the text is too short for statistics, which is its only protection.
4	**Vigenère (m = 7)**	448	Flatter frequencies (IC 0.042); fails once about 60 letters per key column are available.

5	**Mono + columnar (product)**	1792	Frequencies are hidden from the position side and positions from the letter side; the attacker must solve both together.
–	**Hill mod 36**	about 15 known letters	Strongest against frequency analysis (flat histogram) but collapses to any known-plaintext attack because it is linear.

Table 27: Ranking of the schemes by resistance (ciphertext-only attacker, except Hill)

Answer to Task 6

Every scheme was broken. Caesar needs only a few dozen letters; columnar about 112; monoalphabetic and Vigenère about 448; the mono + columnar product about 1792; Hill falls to 4.9 known blocks on average. Attacks work by exploiting preserved letter frequencies (Caesar, mono, transposition), key repetition (Vigenère: Kasiski and IC) and linearity (Hill). The ranking from weakest to strongest is Caesar < columnar < monoalphabetic < Vigenère < product, with Hill the strongest against ciphertext-only statistics and the weakest against known plaintext.

Task 7 – Assess Transposition and Product Ciphers

How columnar transposition diffuses plaintext

In the example of Table 18, the plaintext letters S, E, N, D, L, A, B in the first row are written in columns 1–7 but read out in column order 4, 2, 6, 0, 3, 5, 1 (zero-based). Neighbouring letters of a row therefore land in different column blocks of the ciphertext, separated by one or more whole blocks (4 letters or more in the example, n/7 in general), which removes the letter pairs that digram analysis uses. But no letter is changed: the index of coincidence of the transposition ciphertext is 0.0718 against 0.0719 for the plaintext, and the histogram is identical (Figure 9). Transposition therefore shows at once that it is a rearrangement, not a substitution.

Substitution followed by transposition versus either alone

Measure	Monoalphabetic alone	Columnar alone	Mono + columnar
Letters needed for ≥ 90 % success	448	112	1792
Success at 448 letters	97 %	100 %	67 %
Success at 1,792 letters	100 %	100 %	100 %

Index of coincidence of ciphertext (10,000 letters)	0.0719	0.0718	0.0719
Plain mono attack on the product ciphertext (1,792 letters)	–	–	7.1 % correct
Key space	2^88.4	2^12.3	2^100.7

Table 28: Product cipher compared with its components

The product cipher is clearly stronger than either part: it needs about four times as much ciphertext as the monoalphabetic cipher (1792 against 448 letters) and sixteen times as much as the transposition alone. The attack used here is an alternating search that already knows the number of columns, and the ordinary monoalphabetic attack on the product ciphertext recovers only 7.1 % of the letters, because the trigram statistics it relies on were destroyed by the transposition. The reason for the improvement is the combination: substitution hides which letters are which but keeps their order, transposition hides the order but keeps the letters. Each protects against the attack that works on the other. The gain is limited, however: the notes in this scenario are 200 to 10,000 letters long, and any note longer than about 1,800 letters is broken with a 100 % success rate even in the product form.

Is it practical for rural clinic staff by hand?

Scheme	Operations for a 200-letter note	Letters wrong after one wrong cipher symbol (measured)	Comment
Caesar	200 additions	1.00	Easy; weak
Monoalphabetic	200 table look-ups	1.00	Easy once the table is written
Vigenère	200 additions plus key position	1.00	Easy to lose the key position
Columnar	200 writes + 200 reads	0.99	Grid errors common; one missing letter shifts everything
Mono + columnar	about 600	1 (substitution) + position errors	About three times the effort; feasible with care
Hill 3×3	67 blocks × (9 multiplications + 6 additions) ≈ 1,000	2.89	Five times the effort and one error spoils three letters; unrealistic by hand

Table 29: Manual effort and error propagation

Staff with limited training can use Caesar or monoalphabetic ciphers reliably, but those are the weakest. The product cipher is a reasonable compromise for hand use (two simple steps, one-letter error propagation in the substitution step), while Hill is better reserved for short items such as patient IDs and dosages, and for software.

Lab table: transposition and a classical checksum

A table of 24 lab results (rows of 18 symbols: row number, test code, value, unit, for example "R06POTASS0339MMOLX") plus a checksum row was encrypted with a columnar transposition of width 18 (18! = 6.4×10^15 keys) over the alphabet A–Z, 0–9. Because each matrix row is one table row, an attacker who knows only the width can tamper without the key. Three attacks were simulated, 2,000 trials each. Cell change: replace one ciphertext symbol, which changes exactly one table cell. Row swap: exchange the symbols at matrix rows r1 and r2 inside every ciphertext column block, which swaps two table rows. Cell swap: exchange two symbols of the same matrix row in two column blocks, which swaps two cells of a row. Three protections were compared: none, a sum checksum (sum of symbol values mod 1296, two symbols) and a weighted checksum (Σ i·value mod 1296).

Attack (no key)	No checksum	Sum checksum	Weighted checksum
cell change	0.0 %	97.5 %	96.3 %
row swap	0.0 %	0.0 %	94.3 %
cell swap in a row	0.0 %	0.0 %	100.0 %

Table 30: Share of tampering attempts detected (green ≥ 90 %, red 0 %)

Findings. Without a checksum nothing is detected: a changed cell decrypts to a plausible lab value, and a swapped row is simply a different table. The sum checksum detects a changed cell (97.5 %) but never detects a swapped row or cell, because addition is commutative; the weighted checksum detects swaps (94 % and 100 %) because position matters. The roughly 2–4 % that escape a cell change are the 13 padding symbols of 450 that no checksum covers (2.9 %) and values whose weight shares a factor with 1296.

Where the attacker defeats even the better design. (1) The checksum is unkeyed and public (Kerckhoffs' principle). An insider who holds the transposition key decrypts the table, changes a value, recomputes the checksum and re-encrypts; in the test row R06POTASS0339MMOLX was changed to R06POTASS9999MMOLX and the receiver accepted it as valid. (2) A short checksum has only 1296 values, so a blind random change passes with probability 1/1296. (3) Any linear checksum can be solved from a few known pairs of message and checksum. A checksum detects accidents, not an adversary; a keyed cryptographic MAC or a signature is needed.

Answer to Task 7

Transposition spreads neighbouring letters across the columns but leaves the letter frequencies untouched (IC 0.0718). Substitution followed by transposition needs about four times as much ciphertext to break as substitution alone (1792 against 448 letters) and is the best combination practical by hand, yet it still fails against notes of about 2,000 letters. Classical checksums detect random errors but not deliberate tampering: a sum checksum misses every swap and any checksum falls to an insider who knows the algorithm.

Task 8 – Evaluate Steganography

8.1 Classical encryption of a 256×256 scan

A synthetic 256×256 head-like scan (bone ring, brain tissue, two ventricles, a bright lesion and scanner noise) was encrypted in two classical ways: an intensity shift C = (P + 77) mod 256 (a Caesar cipher on grey levels) and a key-dependent permutation of all 65,536 pixel positions (a transposition). Both were decrypted back to the original exactly (shift: true, permutation: true, both: true).

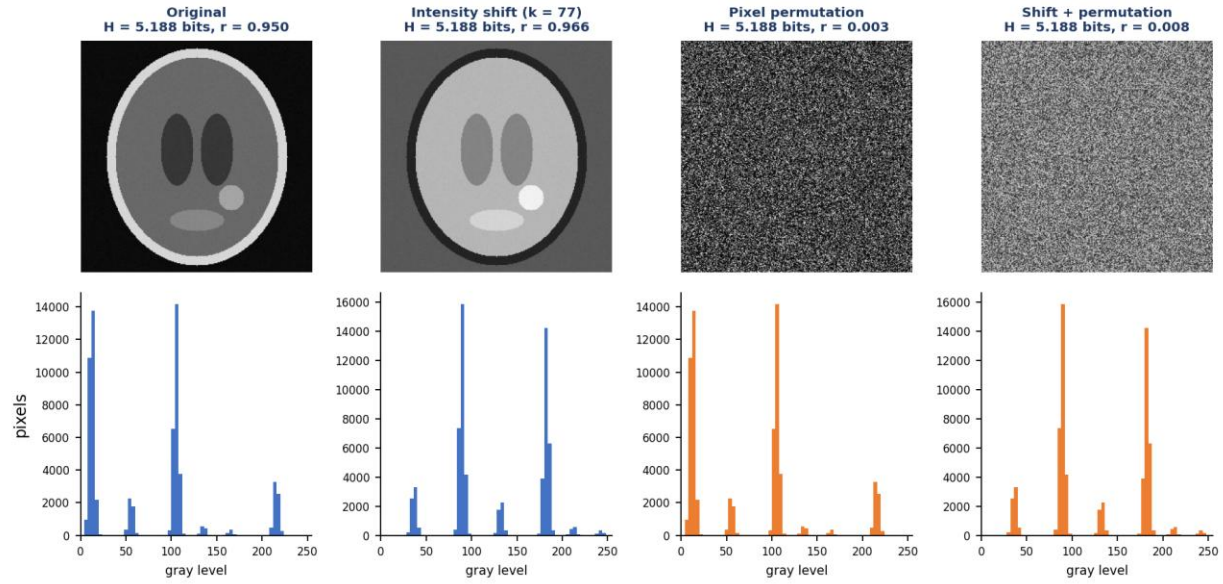


Figure 7: Classical image encryption: images (top) and histograms (bottom)

Image	Histogram entropy (bits)	Neighbour correlation r	Mean horizontal gradient
Original	5.188	0.950	5.33
Intensity shift	5.188	0.966	4.46
Pixel permutation	5.188	0.003	68.27

Shift + permutation	5.188	0.008	57.83

Table 31: Measured properties of the encrypted images

Which leaks more? The intensity shift leaks the entire structure: the head outline, ventricles and lesion are visible (neighbour correlation 0.966, even higher than the original 0.950) and its histogram has the same shape as the original, only moved. The permutation destroys all spatial structure (correlation 0.003) but keeps the histogram exactly, so the proportions of background, tissue and bone, and the number of lesion-valued pixels, remain visible. The histogram entropy is identical in all four images (5.188 bits), a useful warning that entropy of the histogram cannot show whether an image is encrypted: pixel arrangement has to be measured, here by correlation. Shift plus permutation hides the shape, but still shows the shifted histogram, so an attacker who knows typical scan histograms can recover the shift. Neither method is acceptable for real scans.

8.2 Hiding a report in an image and in a circular

Method 1: least-significant-bit (LSB) embedding. A 16-bit length followed by the 8-bit characters of the report "PT-20481 ALLERGY: PENICILLIN. RX AMOX 500MG BD FOR 7 DAYS. REVIEW IN 2 WEEKS." (632 bits in total) replaces the lowest bit of 632 pixels, either in order or in key-shuffled order. Capacity is 1 bit per pixel: 65,536 bits, 8,190 characters. Method 2: text steganography in a hospital circular, where one space between words means 0 and two spaces mean 1; the 654-word circular offers 637 usable bits (79 characters) after the length header. Both were extracted again correctly (image true, text true).

Property	LSB in the scan	Spaces in the circular
Capacity	65,536 bits at 1 bit/pixel	637 bits for 654 words
Imperceptibility	PSNR 71.8 dB for this report; 51.1 dB at 100 % capacity; at most 1 grey level change	77 visible double spaces; looks odd on close reading
Robustness	Destroyed by JPEG at every quality tested (Table below)	Destroyed by any re-typing, copy to HTML or reflow
Detectability	Chi-square test finds a full-capacity random embedding ($p = 0.996$) and a sequential embedding from ≈ 10 %	A scan for double spaces reveals it immediately

Table 32: Comparison of the two steganographic methods

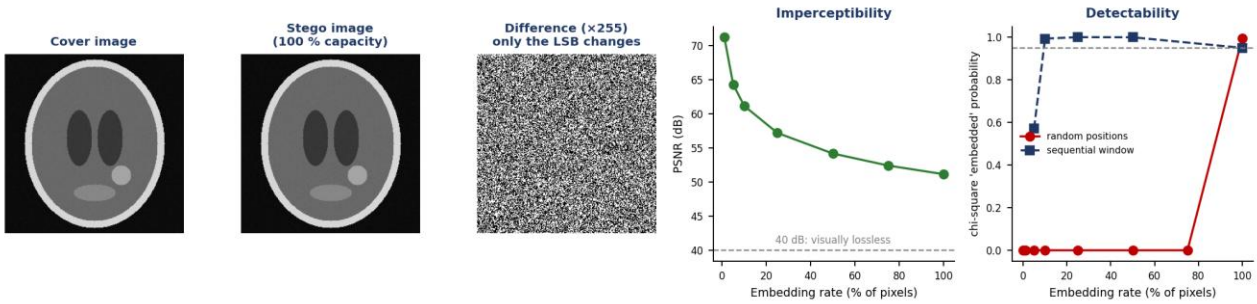


Figure 8: LSB steganography: invisible to the eye (left three images), PSNR falls slowly with payload, detectability depends on the embedding pattern

JPEG quality	95	90	75	50
Bit error rate of the hidden bits	**49.8 %**	**52.1 %**	**53.6 %**	**37.0 %**
Report recovered?	**no**	**no**	**no**	**no**

Table 33: Survival of the LSB message after JPEG compression

Detectability and risks. For random-position embedding the chi-square test (pairs of values 2i, 2i+1) is blind until the image is nearly full (p = 0.000 at 75 %, 0.996 at 100 %), whereas sequential embedding is detected as soon as the first part of the image is tested (5 %: p = 0.57; 10 %: p = 0.99; 25 %: p = 1.00). In a medical image the pixel change is at most one grey level, and the contrast-to-noise ratio of the lesion is unchanged (0.3596 before, 0.3587 after full-capacity embedding), so a radiologist would not see the difference. The danger is different: any change of pixel data breaks the integrity hash of the file and the legal and clinical requirement that a diagnostic image is unaltered; an LSB message would also be removed by the JPEG compression used when scans are sent over rural links.

Encryption versus steganography. Encryption hides the content but shows that a secret exists; steganography hides the existence but not the content once found. Encrypt-then-embed combines them: the report was encrypted with the 36-symbol Vigenère cipher (6DJ0FI5Y25VRR8T242TI…), embedded, extracted and decrypted correctly (true). Hiding adds no integrity or authentication and its capacity is small.

Verdict on Task 8

Steganography is not appropriate for routine patient-record transfer: the capacity is small, the payload does not survive JPEG compression or retyping, it adds no integrity, and altering diagnostic pixels conflicts with medical-image integrity. It is justified only for a narrow case where the very existence of a message must be hidden, and then as encrypt-then-embed in a non-diagnostic area or in file metadata. As encryption, the intensity shift leaks all structure and the pixel permutation leaks the histogram, so neither protects a scan.

Task 9 – Implement and Test

All five schemes, the attacks, the image and text steganography, the lab-table checks, the Bell-LaPadula engine and the experiment drivers are implemented in Python 3 (source in Appendices A–F).

File	Purpose
classical.py (259 lines)	Caesar, monoalphabetic, Vigenère (26 and 36 symbols), columnar transposition, Hill with modular inverse, Hill known-plaintext attack, count of invertible matrices, checksums, IC and frequency functions
attacks.py (305 lines)	Caesar brute force, monoalphabetic frequency and trigram hill climbing, Kasiski, IC key length, Vigenère break, transposition break (Held–Karp), product-cipher break, insider depth attack
imgsec.py (169 lines)	Synthetic scan, shift and permutation image ciphers, entropy, correlation, LSB embedding, PSNR, chi-square, JPEG test, text steganography
labtable.py, blp.py	Lab table with checksums and tampering attacks; Bell-LaPadula engine with compartments, declassification and break-glass
notes.py, riskdata.py	Synthetic clinical notes (no real data); the risk register
run_all.py, worked.py	All experiments (seed 2026) and the worked examples; they write results.json and worked.json
test_all.py (90 lines)	40 correctness and boundary tests with expected and actual output

Table 34: Program files

Identical plaintexts of 100, 1,000, 10,000 and 100,000 characters

Scheme (decrypt(encrypt(m)) = m)	100	1,000	10,000	100,000
Caesar	**PASS**	**PASS**	**PASS**	**PASS**
Monoalphabetic	**PASS**	**PASS**	**PASS**	**PASS**
Vigenère	**PASS**	**PASS**	**PASS**	**PASS**
Columnar	**PASS**	**PASS**	**PASS**	**PASS**
Hill (mod 36)	**PASS**	**PASS**	**PASS**	**PASS**
Mono + columnar	**PASS**	**PASS**	**PASS**	**PASS**

Table 35: Round-trip result for the same plaintext at four sizes (characters)

Edge cases and invalid keys

Input to Hill encryption	Normalised	Ciphertext	Pad	Decrypted	Result
"PT-20481"	PT20481	KJ4OMCLYJ	2	PT20481	**PASS**
"AMOX 500MG"	AMOX500MG	SCCYD6UUK	0	AMOX500MG	**PASS**
"PT-99317 IBUPROFEN 750MG"	PT99317IBUPROFEN750MG	5QBBMFIOXO5752A5C7UUK	0	PT99317IBUPROFEN750MG	**PASS**
"A"	A	U71	2	A	**PASS**
""	(empty)	(empty)	0	(empty)	**PASS**
"9"	9	T6Z	2	9	**PASS**
"PT-20481, AMOX 500MG!"	PT20481AMOX500MG	KJ4OMC1D4WT1QSU0DD	2	PT20481AMOX500MG	**PASS**

Table 36: Hill cipher on IDs, dosages, punctuation, a single character, a digit and empty input

Spaces, hyphens, commas and exclamation marks are removed by the stated convention, so "PT-20481, AMOX 500MG!" becomes PT20481AMOX500MG before encryption. Short blocks (7 symbols, 1 symbol, 10 symbols) are padded and the padding is removed on decryption; an empty message produces an empty ciphertext; a key that is not invertible is rejected.

Expected versus actual output of the test suite

Test	Expected	Actual	Result

Caesar MEET AT NOON, k=4	QIIXEXRSSR	QIIXEXRSSR	**PASS**
Caesar round trip, k=26 (identity)	HELLO	HELLO	**PASS**
Caesar key must be an integer	ValueError	ValueError	**PASS**
Caesar empty input			**PASS**
Mono keyword alphabet	HEALTCRBDFGIJKMNOPQSUVWXYZ	HEALTCRBDFGIJKMNOPQSUVWXYZ	**PASS**
Mono digits/punctuation removed	DOSAGEMG	DOSAGEMG	**PASS**
Mono key with repeated letters	ValueError	ValueError	**PASS**
Vigenere ATTACKATDAWN / LEMON (textbook)	LXFOPVEFRNHR	LXFOPVEFRNHR	**PASS**
Vigenere repeated letters	AAAAAAAAAA	AAAAAAAAAA	**PASS**
Vigenere empty key	ValueError	ValueError	**PASS**
Vigenere-36 AMOX 500MG round trip	AMOX500MG	AMOX500MG	**PASS**
Columnar ZEBRAS textbook example	EVLNXACDTXESEAXROFOXDEECXWIREE	EVLNXACDTXESEAXROFOXDEECXWIREE	**PASS**
Columnar round trip with padding	WEAREDISCOVEREDFLEEATONCE	WEAREDISCOVEREDFLEEATONCE	**PASS**
Columnar key not a permutation	ValueError	ValueError	**PASS**
Hill PT-20481 encrypts to	KJ4OMCLYJ	KJ4OMCLYJ	**PASS**

Hill PT-20481 round trip (7 symbols, pad 2)	PT20481	PT20481	**PASS**
Hill non-multiple length (10 symbols)	AMOX500MG	AMOX500MG	**PASS**
Hill empty input			**PASS**
Hill K * K^-1 = identity	[[1, 0, 0], [0, 1, 0], [0, 0, 1]]	[[1, 0, 0], [0, 1, 0], [0, 0, 1]]	**PASS**
Hill det(K) mod 36 and inverse of det	(7, 31)	(7, 31)	**PASS**
Hill key invertible mod 26 but not mod 36	ValueError	ValueError	**PASS**
Hill known-plaintext attack recovers K	[[1, 1, 3], [1, 2, 1], [2, 8, 1]]	[[1, 1, 3], [1, 2, 1], [2, 8, 1]]	**PASS**
Number of invertible 3x3 matrices mod 2	168	168	**PASS**
100,000 letters: Caesar/mono/Vigenere/ columnar	[True, True, True, True]	[True, True, True, True]	**PASS**
100,000 letters: Hill (mod 36)	True	True	**PASS**
Caesar brute force finds k=11	11	11	**PASS**
Vigenere break finds the key	MEDICAL	MEDICAL	**PASS**
Image shift decrypt = original	True	True	**PASS**
Image permutation decrypt = original	True	True	**PASS**
LSB embed/extract	PT-20481 ALLERGY	PT-20481 ALLERGY	**PASS**

LSB embed/extract (key-shuffled)	RX AMOX 500MG	RX AMOX 500MG	**PASS**
LSB empty message			**PASS**
LSB changes each pixel by at most 1	True	True	**PASS**
LSB message larger than capacity	ValueError	ValueError	**PASS**
Text stego round trip	HI	HI	**PASS**
Text stego circular too short	ValueError	ValueError	**PASS**
BLP nurse reads Secret oncology: deny	DENY	DENY	**PASS**
BLP specialist same level, other compartment	DENY	DENY	**PASS**
BLP doctor writes down to billing: deny	DENY	DENY	**PASS**
BLP trusted declassification	ALLOW (trusted)	ALLOW (trusted)	**PASS**

Table 37: 40 of 40 tests passed (output of python test_all.py)

Testing procedure (repeatable by another person): (1) place all .py files in one folder; (2) run python test_all.py, which prints the table above and exits with an error code if any test fails; (3) run python run_all.py to repeat every experiment with seed 2026 (about two minutes); (4) run python worked.py and python figs_co1.py to regenerate the worked examples and the graphs. Because every random choice uses a fixed seed, repeated runs give identical results except for the timing measurements, which depend on the computer.

Answer to Task 9

All schemes are implemented with decrypt(encrypt(m)) = m for messages of 100, 1,000, 10,000 and 100,000 characters, including digits in patient IDs and dosages (Hill and Vigenère-36), under the stated normalisation rules. 40 of 40 tests pass, covering empty input, spaces and punctuation, repeated letters, non-multiple block

lengths, invalid keys, a textbook Vigenère vector, a textbook columnar example, a known-plaintext key recovery, steganography round trips and Bell-LaPadula decisions.

Task 10 – Record and Interpret Experimental Evidence

Letter histograms

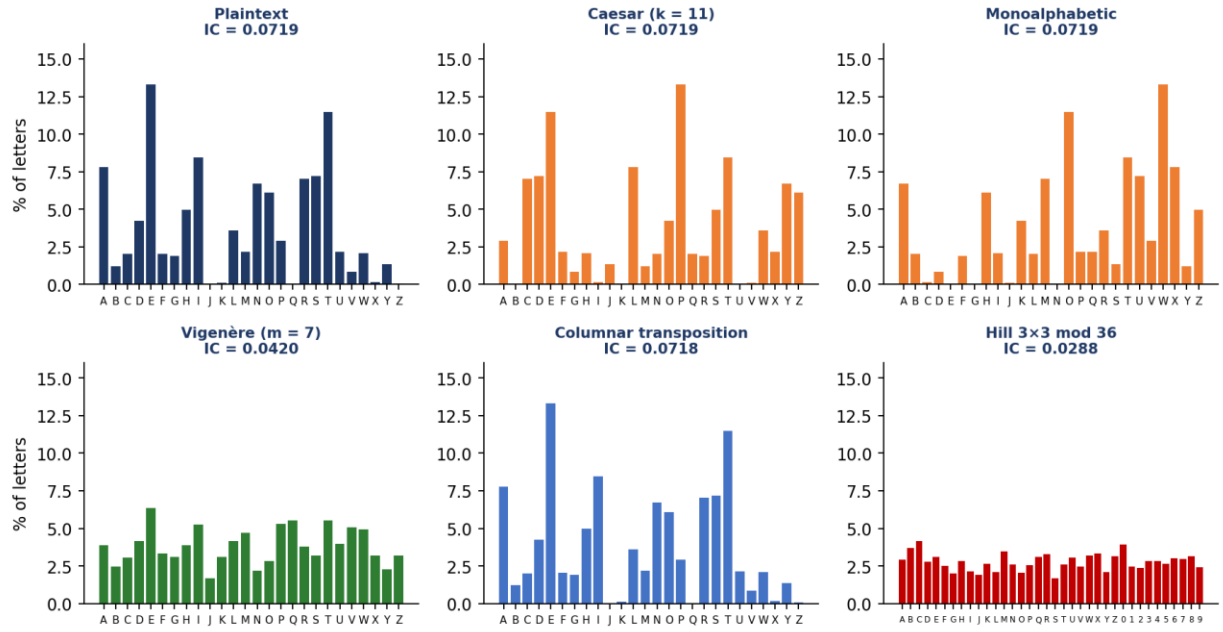


Figure 9: Letter histograms of a 10,000-letter note: plaintext and five ciphertexts

Index of coincidence

Text	IC (measured)	Reading
Plaintext note	0.0719	English-like (natural English ≈ 0.067; the generated notes use fewer words)
Caesar	0.0719	Unchanged: a shift only renames the letters
Monoalphabetic	0.0719	Unchanged: the histogram is the plaintext histogram with the letters renamed
Columnar transposition	0.0718	Unchanged: letters only move
Mono + columnar	0.0719	Unchanged: both parts preserve letter counts

Vigenère, m = 7	0.0420	Lower: seven shifts mix the distributions. Theory: IC ≈ (1/7)·0.0719 + (6/7)·(1/26) = 0.0432
Hill mod 36	0.0288	Close to the random value 1/36 = 0.0278: single-letter statistics are hidden
Random text (26 / 36 symbols)	0.0385 / 0.0278	Reference values

Table 38: Index of coincidence for each scheme (10,000 letters)

Time against message size

Scheme	Enc 100	Enc 1k	Enc 10k	Enc 100k	Dec 100	Dec 1k	Dec 10k	Dec 100k
Caesar	0.014	0.128	1.2	12.2	0.017	0.128	1.2	12.3
Monoalphabetic	0.006	0.035	0.297	3.0	0.006	0.034	0.305	3.5
Vigenere	0.015	0.121	1.1	10.9	0.016	0.125	1.2	10.2
Columnar	0.005	0.034	0.303	2.7	0.008	0.051	0.487	4.1
Hill (mod 36)	0.066	0.539	6.6	50.8	0.067	0.555	5.7	61.5
Mono + columnar	0.015	0.069	0.650	6.8	0.014	0.087	0.822	11.9

Table 39: Encryption and decryption time in milliseconds (this computer)

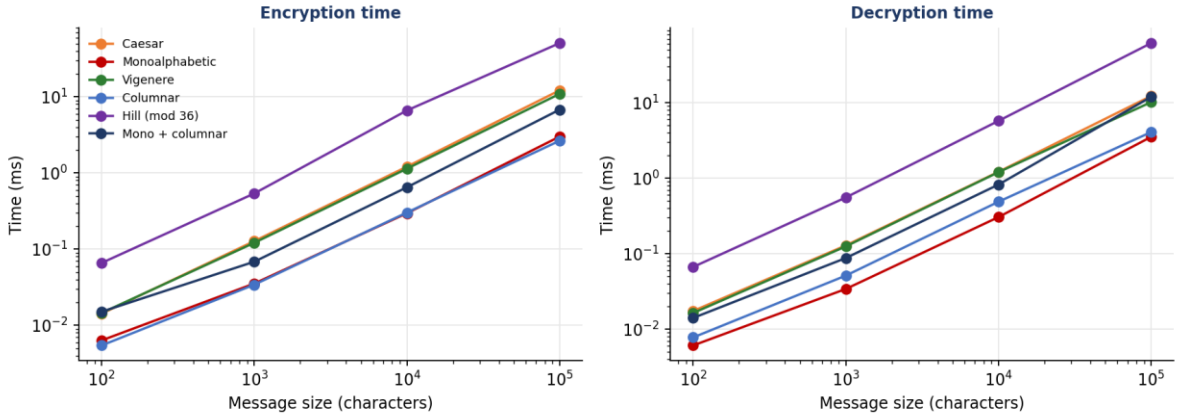


Figure 10: Time against message size: every scheme grows linearly (slope 1 on the log-log graph)

Attack success against ciphertext length

The success rates of Table 26 and Figure 6 (Task 6) are the third measured result: with 112 letters the monoalphabetic attack succeeds in 57 % of trials, with 448 letters in 97 %; Vigenère succeeds in 70 % at 224 letters and 100 % at 448.

Interpretation: theory against results

Prediction (Tasks 5 and 6)	Measurement	Agreement
Shift, substitution and transposition preserve the letter-frequency pattern, so IC stays at the plaintext value	IC 0.0719, 0.0719, 0.0718 against 0.0719	**Exact**
Vigenère with period 7 gives IC ≈	0.0432 predicted, 0.0420 measured	**Within 3 %**
Hill's block mixing flattens the histogram towards 1/36 = 0.0278	0.0288	**Agrees; histogram flat**
Kasiski distances are multiples of the key length	7 is the most common divisor (118 votes), then 2, 14, 3, 21	**Agrees; 2 and 3 are common divisors of any distance**
More ciphertext gives more statistical power: success rises with length	Every scheme rises to 100 % between 56 and 1,792 letters	**Agrees**
A bigger key space is not a stronger cipher	2^88.4 keys broken at 448 letters; Caesar's 25 keys at 56	**Confirmed**
Encryption time is O(n)	Time grows by about 1,000 when the size grows by 1,000	**Agrees**

Table 40: Theory-to-result discussion

Limits of the evidence. The notes are synthetic and have a narrower vocabulary than real notes, so the attack thresholds are a guide to the ordering of the schemes and not exact lengths for real clinical text; the n-gram model of the attacker was learnt from the same kind of synthetic text, which favours the attacker (the correct assumption for a defender). Timing was measured once on one computer; the shape of the curves, not the milliseconds, is the result. Speed is irrelevant to the choice, because every scheme takes under 50 ms for 100,000 characters; strength, not speed, decides.

Answer to Task 10

Histograms and IC show that Caesar, monoalphabetic and transposition ciphertexts keep the plaintext's IC (0.0719), Vigenère (m = 7) lowers it to 0.0420 as predicted (0.0432), and Hill brings it to 0.0288 (random 0.0278). Time grows linearly for all schemes (Hill slowest, 48 ms for 100,000 characters). Attack success rises with ciphertext length for every scheme, matching the theory of Tasks 5 and 6.

Task 11 – Recommend and Defend the Final Design

Alternatives considered

Option	Evidence against / for	Fits rural staff?	Verdict
Caesar	25 keys; broken at ≤ 56 letters	Yes	**Reject**
Monoalphabetic	Broken at 448 letters; frequency preserved	Yes	**Reject**
Vigenère (long key)	Needs ≈ 60 letters per key column; with 12 letters ≈ 700 letters; repeated key leaks to depth attacks	Yes, key tracking needed	**Use as one layer**
Columnar transposition	Alone broken at 112 letters, but removes digram structure	Yes, careful gridding	**Use as second layer**
Mono + columnar product	1792 letters needed (4× mono)	Moderate effort	**Good but limited**
Hill mod 36	Flat histogram but ≈ 5 known blocks give the key	Heavy by hand	**Reject for notes**
Vigenère-36 (A–Z, 0–9) + columnar	Also hides digits of IDs and dosages; as above	Yes	**Recommended (classical)**

Table 41: Alternatives against the evidence

Recommended design (original, before the insider incident)

Layer	Choice	Why (evidence)
Notes and IDs	Vigenère over 36 symbols with a random key of at least 12 symbols, then columnar transposition with at least 8 columns, applied to the whole message including IDs and dosages	Product cipher needs 1792 letters against 448 for mono (Task 7); digits must not be sent in clear
Integrity	Weighted checksum inside the message, serial number and date, and two-person verification of every dose	Weighted checksum detects swaps and 96 % of cell changes (Task 7); it cannot stop an adversary
Access control	Bell-LaPadula with compartments; separate keys per compartment (GEN, ONC, PSY, HIV) so that cryptographic access matches policy; break-glass with audit	Task 3: compartments stop same-level cross access
Key distribution	Hub and spoke: the central hospital holds one key per clinic and compartment; keys issued on printed cards by courier and replaced monthly	25 keys instead of 325 for a full mesh; one clinic compromise exposes only that clinic
Scans	No steganography in diagnostic images; encrypt the file as a whole	Task 8: stego fragile, alters pixels

Table 42: Recommended design

Residual risks and an honest limit

- Classical ciphers give no integrity, authentication or non-repudiation (Task 2); the checksum and procedures reduce accidents, not forgery.
- Even the recommended layered design is broken by enough ciphertext: roughly 60 letters per key column for Vigenère and 4× more with the product, so notes above about 1,000–2,000 letters remain at risk, and the insider scenario of Task 12 breaks it outright.
- Key distribution: 25 keys for hub-and-spoke is manageable, but monthly cards for 4 compartments × 25 clinics = 100 keys require trained couriers and a lost card is a breach.
- Human error: a single wrong symbol is corrected by the receiver only if the length is known; the transposition breaks all following letters if one is lost.

Judgement: are classical ciphers acceptable for patient data at all? No. Every scheme in this report was broken with a few hundred to a few thousand letters, and the strongest (Hill) falls to a handful of known blocks. For a real network the data must be protected with modern authenticated encryption (for example AES-256-GCM), TLS 1.3 for transport, digital signatures for orders and prescriptions, certificates and a key-management system, none of which is classical. The classical design above is justified only as a teaching exercise or, with strict limits on message length and key lifetime, as a short-term stopgap.

Answer to Task 11

Recommended (classical) design: Vigenère-36 with a random key of at least 12 symbols followed by an 8-column or wider transposition for every message, a weighted checksum with two-person verification for doses, Bell-LaPadula with compartments and one key per clinic and compartment, hub-and-spoke key distribution by monthly courier, and no steganography on diagnostic scans. The defence rests on the measured ordering of Tasks 6, 7 and 10; residual risks (no integrity or authentication, breakable long notes, courier key distribution) are stated, and for real patient data the design should be replaced by modern authenticated encryption.

Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt

Changed requirement: a clinic insider has copied hundreds of ciphertexts under the same key; the network also carries hidden scan messages (Task 8). Each part below answers the corresponding question.

12A. Impact analysis

- **What has changed.** The attacker is no longer a stranger seeing a few messages: he holds hundreds of messages, all under one key, all beginning with the same predictable header (PATIENT ID … DIAGNOSIS … PRESCRIPTION). Each message is a known-plaintext sample for the header and one more aligned sample for a depth attack.
- **Assumptions that are no longer safe.** (1) That keys remain secret for a month and across 25 sites: one reused key is exposed by volume alone. (2) That messages are independent: they share key and structure. (3) That the attacker has only a few messages: the measured threshold is about 15 (Figure 11). (4) That predictable headers are harmless: they hand the attacker the key stream (12B). (5) That the protections for confidentiality also protect integrity and authenticity: they do not.

Code	Attack	L	I	L×I	What changed

A7	Insider copies ciphertexts and recovers the key	5	5	**25**	Hundreds of known-header messages under one key: key recovered, every message readable and forgeable.
A3	Modification of prescription / dosage	5	5	**25**	Insider with the key can alter any message and re-encrypt it; no check can stop this.
A4	Masquerade as a doctor	5	5	**25**	The key is shared by all 25 clinics, so a clinic insider can impersonate any other site.
A1	Eavesdropping on the public network	5	5	**25**	All past and future traffic of all clinics becomes readable.
A5	Replay of old orders	5	4	**20**	Insider can replay and also craft fresh valid orders.
A6	Denial of service on the records system	3	4	**12**	Unchanged by the key compromise.
A2	Traffic analysis	4	2	**8**	Unchanged.

Table 43: Risk register after the insider incident (right panel of Figure 1); ties are listed with the root cause (A7) first

A new attack A7 (insider recovers the key from collected ciphertexts) becomes the root risk. Eavesdropping (A1) jumps from likelihood 5 × impact 3 = 15 to 5 × 5 = 25 because the insider can decrypt everything, past and future, of all clinics that share the key. Modification (A3) and masquerade (A4) also reach 25: an attacker who knows the key can create any message. The high volume of known-format messages changes the ranking because every new message lowers the attacker's work instead of raising it.

12B. Evaluation of the original design against the new capabilities

Four tests were run against the recommended design of Task 11 (and its parts):

Test	Result	Verdict
Known header against Vigenère, one message	Ciphertext start WOLXMGTTFPKH…; header PATIENTIDPTD…; subtracting gives the key stream HOSPITALCARE, equal to the true key HOSPITALCARE (match = true)	**Key recovered from ONE message**

Known header against Hill	Six header blocks gave K exactly (Task 6); on average 4.9 blocks suffice	**Key recovered**
Depth attack on Vigenère (key 12) + columnar (7 columns), ciphertext only, same key	1 msg: 14 %; 2 msg: 11 %; 5 msg: 8 %; 10 msg: 19 %; 15 msg: 84 %; 20 msg: 89 %; 30 msg: 90 %; 50 msg: 91 %; 100 msg: 92 %; 300 msg: 92 %	**Fails below ≈ 10 messages; recovers 84–92 % of all letters from ≥ 15**
Bell-LaPadula against the insider	BLP controls who may ask the system for a record. A message is decrypted with a shared key outside the reference monitor, so once the key is known the policy has no effect. A Confidential-level clinic insider who learns the shared key can read Secret traffic.	**Holds only for system-mediated access**

Table 44: Evaluation of the original design

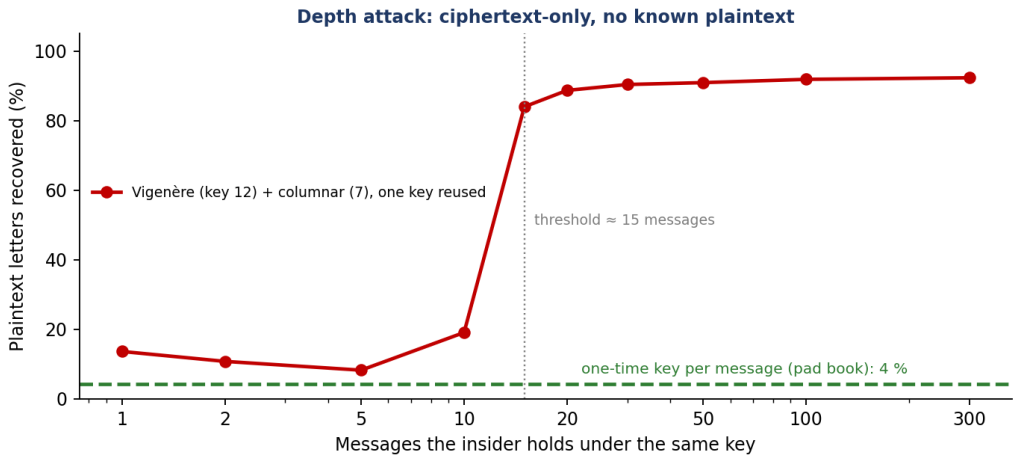


Figure 11: Depth attack on the layered scheme: success jumps at about 15 messages with the same key; a fresh key per message leaves 4 % (chance)

How the depth attack works. In Vigenère followed by transposition every ciphertext position has one fixed shift, because the key position and the transposition are the same in every message. A Caesar frequency attack on that position across the collected messages recovers the shift. After removing the shifts, only the transposition remains, and the column order is found from the bigram statistics summed over all messages. No known plaintext and no knowledge of the key are needed; 92 % of all letters were recovered with 300 messages, and with a fresh random key per message the same attack recovers 4 % (chance level). The key spaces involved are 26^12 × 7! ≈ 2^68.7; the attack ignores them.

Scans. The 256×256 scan carries a 632-bit hidden report (Task 8) that is invisible (PSNR 71.8 dB), but the chi-square test finds sequential embeddings from about 10 % capacity and the JPEG compression used for rural links destroys it, so hidden scan messages are neither safe from the insider nor reliable.

12C. Adaptation or redesign

Decision: REDESIGN the key handling and message format; MODIFY the access policy; RETIRE the weak ciphers. Specifically:

Change	Effect on key space	Effect on attack effort	Cost
1 One-time key per message (pad book: random letters as long as the message, used once and destroyed)	26^L per message (L = 336 letters: 2^1,579), key never reused	Depth attack recovers 4 % = chance; ciphertext gives no information	36.5 MB of key per clinic per year; 913 MB for 25 clinics
2 If pads are impossible: rotate the key after at most 10 messages	Unchanged per key	Stays below the ≈ 15-message threshold (13 %, 11 %, 8 %, 19 % at 1, 2, 5, 10 messages)	10 new keys per clinic per day
3 Remove predictable headers: put the header inside the encrypted body after random padding of varying length	None	Removes the crib for known-plaintext (one-message key recovery and Hill KPA)	About 20 extra letters
4 Separate keys for each clinic and compartment (100 keys)	One shared key → 100 independent keys	A compromise exposes one clinic and one compartment; matches Bell-LaPadula	Key management of 100 keys
5 Encrypt, then embed (only where hiding existence is required)	As the cipher	Protects content if the cover is found	Small capacity; not for diagnostic pixels
6 Integrity by procedure: serial number, date, two-person dose check, weighted checksum; plan to	–	Replay and accidental errors are caught; deliberate forgery by a key holder is not	Staff time

move to modern authentication			

Table 45: Redesign decisions and their effect

Why the one-time pad? It is the only classical technique with provable secrecy: if the key is truly random, as long as the message and never reused, every plaintext of the same length is equally likely, and the key-space argument becomes meaningful (2^1579 for a 336-letter message). The depth attack's 4 % is exactly what chance gives. Its price is key volume and the discipline of single use: reusing a pad even once turns it into a two-time pad and recovers the plaintext. It still gives no integrity (it is as malleable as Vigenère, Task 2) and no authentication, so procedural controls remain necessary.

12D. Evidence-based defence

Evidence	What it shows for the final design
Key-space mathematics (Task 5)	Key spaces are either small (Caesar 2^4.6, columnar 7 columns 2^12.3) or irrelevant to the attack (monoalphabetic 2^88.4 broken at 448 letters). Only the one-time pad's key grows with the message.
Cryptanalysis results (Tasks 6, 7, 10)	All schemes broken: Caesar at 56, columnar 112, mono 448, Vigenère 448, product 1792 letters; Hill with 4.9 known blocks.
Insider and key reuse (12B)	A single message plus the header gives the Vigenère key; ≥ 15 messages give 84–92 % of all letters of the layered scheme; a fresh key per message gives 4 %.
Key-management cost (12C)	A pad book costs 36.5 MB per clinic per year (100 notes of 1,000 letters a day); hub-and-spoke needs 25 keys against 325 for full mesh; separate compartments need 100.
Usability (Task 7)	Pad use by hand is one addition per letter, as simple as Caesar, but needs a courier chain and strict single use; Hill by hand is about 5 times the work.
What classical techniques cannot guarantee	Integrity, authentication, non-repudiation, availability, forward secrecy, protection against a key holder. These need MACs and digital signatures, which are not classical.

Table 46: Evidence for the final decision

Answer to Task 12

A: the insider and the volume of messages with a fixed header raise eavesdropping, modification and masquerade to 25 of 25 and add the root risk A7. B: the original design fails: the key is recovered from one

message plus the header, Hill from about 5 blocks, and the layered Vigenère + transposition leaks 92 % of its letters from 100 messages (84 % from 15). C: redesign with a one-time key per message (4 % recovery, chance), separate keys per clinic and compartment, headers inside the encrypted body, procedural integrity, and no hidden messages in diagnostic scans. D: this is the strongest classical answer, but because classical techniques cannot provide integrity, authentication or non-repudiation, a real deployment must move to modern authenticated encryption and signatures.

Appendix A – classical.py

classical.py

```
"""
classical.py - classical ciphers used in the report (CSA51, CO1 Assignment 1).

Conventions (stated once, used everywhere):
  * Letter ciphers (Caesar, monoalphabetic, Vigenere, columnar transposition) work on the 26 letters A-Z.
    Input is converted to upper case and every character that is not A-Z (spaces, digits, punctuation)
    is removed by normalize().  decrypt(encrypt(m)) therefore equals normalize(m).
  * The Hill cipher and the Vigenere-36 variant use the 36-symbol alphabet A-Z then 0-9 (A=0 ... Z=25, 0=26
... 9=35).
    Any other character is removed; short blocks are padded with 'X'.
  * Invalid keys raise ValueError (never fail silently).
"""
import math
import random
import string
from itertools import combinations

A26 = string.ascii_uppercase
A36 = string.ascii_uppercase + string.digits



# ---------------------------------------------------------------- helpers
def normalize(text, alphabet=A26):
    """Upper-case the text and keep only symbols of the alphabet."""
    return "".join(ch for ch in text.upper() if ch in alphabet)


def ic(text, alphabet=A26):
    """Index of coincidence of a text."""
    n = len(text)
```

```
    if n < 2:
        return 0.0
    return sum(text.count(ch) * (text.count(ch) - 1) for ch in alphabet) / (n * (n - 1))


def freq(text, alphabet=A26):
    n = max(len(text), 1)
    return [text.count(ch) / n for ch in alphabet]


# ---------------------------------------------------------------- Caesar
def check_caesar_key(k):
    if not isinstance(k, int) or isinstance(k, bool):
        raise ValueError("Caesar key must be an integer")
    return k % 26


def caesar_encrypt(text, k):
    k = check_caesar_key(k)
    return "".join(A26[(A26.index(c) + k) % 26] for c in normalize(text))


def caesar_decrypt(text, k):
    return caesar_encrypt(text, -check_caesar_key(k))


# ---------------------------------------------------------------- monoalphabetic substitution
def check_mono_key(key):
    if len(key) != 26 or sorted(key.upper()) != list(A26):
        raise ValueError("monoalphabetic key must be a permutation of the 26 letters")
    return key.upper()
```

```
def keyword_key(keyword):
    """Keyword alphabet: keyword (duplicates removed) followed by the unused letters."""
    kw = "".join(dict.fromkeys(normalize(keyword)))
    if not kw:
        raise ValueError("keyword must contain at least one letter")
    return kw + "".join(c for c in A26 if c not in kw)


def random_mono_key(rng):
    letters = list(A26)
    rng.shuffle(letters)
    return "".join(letters)


def mono_encrypt(text, key):
    table = str.maketrans(A26, check_mono_key(key))
    return normalize(text).translate(table)


def mono_decrypt(text, key):
    table = str.maketrans(check_mono_key(key), A26)
    return normalize(text).translate(table)


# ---------------------------------------------------------------- Vigenere (any alphabet)
def check_vig_key(key, alphabet):
    k = normalize(key, alphabet)
    if not k:
        raise ValueError("Vigenere key must contain at least one symbol of the alphabet")
    return k
```

```
def vigenere_encrypt(text, key, alphabet=A26):
    key = check_vig_key(key, alphabet)
    m = len(alphabet)
    idx = {c: i for i, c in enumerate(alphabet)}
    kv = [idx[c] for c in key]
    return "".join(alphabet[(idx[c] + kv[i % len(kv)]) % m] for i, c in enumerate(normalize(text, alphabet)))


def vigenere_decrypt(text, key, alphabet=A26):
    key = check_vig_key(key, alphabet)
    m = len(alphabet)
    idx = {c: i for i, c in enumerate(alphabet)}
    kv = [idx[c] for c in key]
    return "".join(alphabet[(idx[c] - kv[i % len(kv)]) % m] for i, c in enumerate(normalize(text, alphabet)))


# ---------------------------------------------------------------- columnar transposition
def check_perm(perm):
    if sorted(perm) != list(range(len(perm))) or len(perm) < 2:
        raise ValueError("transposition key must be a permutation of 0..k-1 with k >= 2")
    return list(perm)


def keyword_perm(keyword):
    """Column order from a keyword: columns are read in alphabetical order of the keyword letters."""
    kw = normalize(keyword)
    order = sorted(range(len(kw)), key=lambda i: (kw[i], i))
    return order          # order[j] = index of the column that is read j-th


def pad_to(text, k, pad="X"):
```

```
    return text + pad * (-len(text) % k)


def columnar_encrypt(text, perm, alphabet=A26):
    """Write the text row by row into k columns, read the columns in the order given by perm."""
    perm = check_perm(perm)
    k = len(perm)
    t = pad_to(normalize(text, alphabet), k)
    return "".join(t[c::k] for c in perm)


def columnar_decrypt(cipher, perm, orig_len=None):
    perm = check_perm(perm)
    k = len(perm)
    if len(cipher) % k:
        raise ValueError("ciphertext length must be a multiple of the key length")
    rows = len(cipher) // k
    cols = {}
    for j, c in enumerate(perm):
        cols[c] = cipher[j * rows:(j + 1) * rows]
    plain = "".join(cols[c][r] for r in range(rows) for c in range(k))
    return plain[:orig_len] if orig_len is not None else plain


# ---------------------------------------------------------------- modular matrices and Hill cipher
def det(M, m=None):
    n = len(M)
    if n == 1:
        d = M[0][0]
    elif n == 2:
        d = M[0][0] * M[1][1] - M[0][1] * M[1][0]
    else:
```

```
    d = sum(((-1) ** j) * M[0][j] * det([row[:j] + row[j + 1:] for row in M[1:]]) for j in range(n))
  return d % m if m else d


def adjugate(M, m):
  n = len(M)
  if n == 1:
    return [[1 % m]]
  adj = [[0] * n for _ in range(n)]
  for i in range(n):
    for j in range(n):
      minor = [row[:j] + row[j + 1:] for r, row in enumerate(M) if r != i]
      adj[j][i] = ((-1) ** (i + j)) * det(minor) % m
  return adj


def mat_inv(M, m):
  """Inverse of M modulo m via adjugate and the modular inverse of det (exists iff gcd(det, m) = 1)."""
  d = det(M, m)
  if math.gcd(d, m) != 1:
    raise ValueError(f"matrix is not invertible mod {m}: det = {d}, gcd(det, {m}) = {math.gcd(d, m)}")
  di = pow(d, -1, m)
  return [[(di * x) % m for x in row] for row in adjugate(M, m)]


def mat_mul(A, B, m):
  return [[sum(A[i][t] * B[t][j] for t in range(len(B))) % m for j in range(len(B[0]))] for i in range(len(A))]


def count_invertible(n, m):
  """Number of invertible n x n matrices mod m = product over prime powers p^e || m of
    p^((e-1) n^2) * prod_{i<n} (p^n - p^i)."""
```

```
    total, x, p = 1, m, 2
    fac = {}
    while x > 1:
        while x % p == 0:
            fac[p] = fac.get(p, 0) + 1
            x //= p
        p += 1
    for p, e in fac.items():
        part = p ** ((e - 1) * n * n)
        for i in range(n):
            part *= p ** n - p ** i
        total *= part
    return total


def hill_encrypt(text, K, alphabet=A36, pad="X"):
    """C = K * P (mod m) on column blocks of n symbols.  Returns (ciphertext, number of pad symbols)."""
    m, n = len(alphabet), len(K)
    mat_inv(K, m)                          # raises ValueError for a non-invertible key
    idx = {c: i for i, c in enumerate(alphabet)}
    t = normalize(text, alphabet)
    padn = -len(t) % n
    t += pad * padn
    out = []
    for b in range(0, len(t), n):
        p = [idx[c] for c in t[b:b + n]]
        out += [alphabet[sum(K[i][j] * p[j] for j in range(n)) % m] for i in range(n)]
    return "".join(out), padn


def hill_decrypt(cipher, K, padn=0, alphabet=A36):
    m, n = len(alphabet), len(K)
```

```
    Ki = mat_inv(K, m)
    idx = {c: i for i, c in enumerate(alphabet)}
    if len(cipher) % n:
        raise ValueError("ciphertext length must be a multiple of the block size")
    out = []
    for b in range(0, len(cipher), n):
        c = [idx[x] for x in cipher[b:b + n]]
        out += [alphabet[sum(Ki[i][j] * c[j] for j in range(n)) % m] for i in range(n)]
    s = "".join(out)
    return s[:len(s) - padn] if padn else s


def hill_kpa(pairs, n, alphabet=A36):
    """Known-plaintext attack: pairs = list of (plain_block, cipher_block) strings of n symbols.
       Tries every choice of n pairs whose plaintext matrix is invertible and returns K = C * P^-1 (mod m)."""
    m = len(alphabet)
    idx = {c: i for i, c in enumerate(alphabet)}
    for combo in combinations(range(len(pairs)), n):
        P = [[idx[pairs[b][0][r]] for b in combo] for r in range(n)]      # plaintext blocks as columns
        C = [[idx[pairs[b][1][r]] for b in combo] for r in range(n)]
        try:
            return mat_mul(C, mat_inv(P, m), m), combo
        except ValueError:
            continue
    return None, None


# ---------------------------------------------------------------- classical checksums (Task: lab table)
def checksum_sum(text, alphabet=A36):
    """Unweighted sum of the symbol values, two base-36 characters (mod 1296)."""
    v = sum(alphabet.index(c) for c in text) % (36 * 36)
    return alphabet[v // 36] + alphabet[v % 36]
```

```
def checksum_weighted(text, alphabet=A36):
    """Position-weighted sum  sum((i+1) * value_i)  mod 1296, two characters."""
    v = sum((i + 1) * alphabet.index(c) for i, c in enumerate(text)) % (36 * 36)
    return alphabet[v // 36] + alphabet[v % 36]
```

Appendix B – attacks.py

attacks.py

```
"""
attacks.py - cryptanalysis tools used in the report.

Attacker model (stated in the report): the attacker knows the algorithm (Kerckhoffs' principle), the clinic-note
domain and public English letter statistics.  N-gram statistics for the substitution and transposition attacks are
learned from a separate synthetic clinical corpus (different random seed from every test message).
"""
import math
import random
from collections import Counter
from itertools import combinations

import numpy as np

import classical as C
import notes

# public English letter frequencies (percent), used for the Caesar and Vigenere attacks
ENGLISH = [8.17, 1.29, 2.78, 4.25, 12.70, 2.23, 2.02, 6.09, 6.97, 0.15, 0.77, 4.03, 2.41,
           6.75, 7.51, 1.93, 0.10, 5.99, 6.33, 9.06, 2.76, 0.98, 2.36, 0.15, 1.97, 0.07]
ENG = np.array(ENGLISH) / 100.0


def _build_model(seed=777, n=400000):
    t = notes.corpus(seed, n)
    a = np.frombuffer(t.encode(), dtype=np.uint8).astype(np.int64) - 65
    c1 = np.bincount(a, minlength=26) + 0.5
    c2 = np.bincount(a[:-1] * 26 + a[1:], minlength=676) + 0.5
    c3 = np.bincount(a[:-2] * 676 + a[1:-1] * 26 + a[2:], minlength=17576) + 0.5
    return np.log(c1 / c1.sum()), np.log(c2 / c2.sum()).reshape(26, 26), np.log(c3 / c3.sum())
```

```
LP1, LP2, LP3 = _build_model()
LP3F = LP3.ravel()


def to_arr(s):
    return np.frombuffer(s.encode(), dtype=np.uint8).astype(np.int64) - 65


def to_str(a):
    return (np.asarray(a, dtype=np.uint8) + 65).tobytes().decode()


def accuracy(guess, truth):
    n = min(len(guess), len(truth))
    return sum(1 for i in range(n) if guess[i] == truth[i]) / max(len(truth), 1)


# ---------------------------------------------------------------- Caesar: brute force, scored by chi-square
def chi_square(counts, n, expected=ENG):
    e = expected * n
    return float(((counts - e) ** 2 / np.maximum(e, 1e-9)).sum())


def caesar_break(ct):
    a = to_arr(ct)
    best = None
    for k in range(26):
        cnt = np.bincount((a - k) % 26, minlength=26)
        x = chi_square(cnt, len(a))
        if best is None or x < best[0]:
```

```
            best = (x, k)
    return best[1], C.caesar_decrypt(ct, best[1])


# ---------------------------------------------------------------- monoalphabetic: frequency start + hill climbing
def mono_break(ct, rng, restarts=6):
    c = to_arr(ct)
    n = len(c)
    tri = c[:-2] * 676 + c[1:-1] * 26 + c[2:]
    u, cnt = np.unique(tri, return_counts=True)
    ta, tb, tc = u // 676, (u // 26) % 26, u % 26

    def score(p):
        return float((cnt * LP3F[p[ta] * 676 + p[tb] * 26 + p[tc]]).sum())

    cipher_rank = np.argsort(-np.bincount(c, minlength=26))       # cipher letters, most frequent first
    plain_rank = np.argsort(-LP1)                                 # plain letters, most frequent first
    start = np.zeros(26, dtype=np.int64)
    start[cipher_rank] = plain_rank                               # frequency-analysis guess
    best_p, best_s = start.copy(), score(start)
    for r in range(restarts):
        p = start.copy()
        for _ in range(0 if r == 0 else 6 + 2 * r):               # perturb the frequency guess
            i, j = rng.sample(range(26), 2)
            p[i], p[j] = p[j], p[i]
        cur = score(p)
        improved = True
        while improved:
            improved = False
            for i in range(25):
                for j in range(i + 1, 26):
                    p[i], p[j] = p[j], p[i]
```

```
                s = score(p)
                if s > cur + 1e-9:
                    cur, improved = s, True
                else:
                    p[i], p[j] = p[j], p[i]
        if cur > best_s:
            best_p, best_s = p.copy(), cur
    key_cipher_to_plain = "".join(C.A26[x] for x in best_p)       # cipher letter i -> plain letter
    return key_cipher_to_plain, to_str(best_p[c])



# ---------------------------------------------------------------- Vigenere: Kasiski, index of coincidence, columns
def kasiski(ct, ngram=3, maxlen=24):
    """Distances between repeated n-grams, and how often each key length 2..maxlen divides them."""
    pos = {}
    for i in range(len(ct) - ngram + 1):
        pos.setdefault(ct[i:i + ngram], []).append(i)
    dists = [b - a for p in pos.values() if len(p) > 1 for a, b in zip(p, p[1:])]
    votes = Counter()
    for d in dists:
        for m in range(2, maxlen + 1):
            if d % m == 0:
                votes[m] += 1
    return len(dists), votes



def avg_ic_by_length(ct, maxm=24):
    out = {}
    for m in range(1, maxm + 1):
        cols = [ct[i::m] for i in range(m)]
        out[m] = sum(C.ic(c) for c in cols) / m
    return out
```

```
def guess_key_length(ct, maxm=24, tol=0.004):
    ics = avg_ic_by_length(ct, maxm)
    best = max(ics.values())
    return min(m for m, v in ics.items() if v >= best - tol), ics


def vigenere_break(ct, maxm=24):
    m, ics = guess_key_length(ct, maxm)
    key = ""
    for i in range(m):
        col = ct[i::m]
        k, _ = caesar_break(col) if len(col) > 1 else (0, "")
        key += C.A26[k]
    return key, C.vigenere_decrypt(ct, key), m


# ---------------------------------------------------------------- columnar transposition: column-adjacency + Held-Karp
def best_column_order(chunks):
    """chunks: list of k equal-length strings (ciphertext columns).  Returns the left-to-right order maximising
       the summed bigram log-likelihood of horizontally adjacent letters (Held-Karp dynamic programme)."""
    k = len(chunks)
    arr = [to_arr(x) for x in chunks]
    W = np.zeros((k, k))
    for a in range(k):
        for b in range(k):
            if a != b:
                W[a, b] = LP2[arr[a], arr[b]].sum()
    NEG = -1e18
    dp = [[NEG] * k for _ in range(1 << k)]
```

```
    par = [[-1] * k for _ in range(1 << k)]
    for i in range(k):
        dp[1 << i][i] = 0.0
    for mask in range(1, 1 << k):
        for last in range(k):
            v = dp[mask][last]
            if v == NEG or not (mask >> last) & 1:
                continue
            for nxt in range(k):
                if (mask >> nxt) & 1:
                    continue
                nm = mask | (1 << nxt)
                s = v + W[last, nxt]
                if s > dp[nm][nxt]:
                    dp[nm][nxt], par[nm][nxt] = s, last
    full = (1 << k) - 1
    last = max(range(k), key=lambda i: dp[full][i])
    order, mask = [], full
    while last != -1:
        order.append(last)
        mask, last = mask ^ (1 << last), par[mask][last]
    return order[::-1], dp[full][max(range(k), key=lambda i: dp[full][i])]


def transposition_break(ct, kmax=9):
    n = len(ct)
    best = None
    for k in range(2, kmax + 1):
        if n % k:
            continue
        rows = n // k
        chunks = [ct[j * rows:(j + 1) * rows] for j in range(k)]
```

```
        order, _ = best_column_order(chunks)
        plain = "".join(chunks[o][r] for r in range(rows) for o in order)
        a = to_arr(plain)
        s = float(LP2[a[:-1], a[1:]].sum())
        if best is None or s > best[0]:
            best = (s, k, order, plain)
    return best[1], best[2], best[3]



# ---------------------------------------------------------------- product cipher (substitution then transposition)
def product_break(ct, k, rng, rounds=5, restarts=4):
    """Alternate between (a) best column order for the current substitution guess and (b) hill climbing of the
       substitution for the current column order.  The number of columns k is given to the attacker."""
    n = len(ct)
    if n % k:
        raise ValueError("length not a multiple of k")
    rows = n // k
    chunks = [ct[j * rows:(j + 1) * rows] for j in range(k)]
    c_all = to_arr(ct)
    cipher_rank = np.argsort(-np.bincount(c_all, minlength=26))
    plain_rank = np.argsort(-LP1)
    perm = np.zeros(26, dtype=np.int64)
    perm[cipher_rank] = plain_rank
    order = list(range(k))
    best = (-1e18, None, None)
    for rr in range(restarts):
        p = perm.copy()
        for _ in range(0 if rr == 0 else 8):
            i, j = rng.sample(range(26), 2)
            p[i], p[j] = p[j], p[i]
        order_cur = list(range(k))
        for _ in range(rounds):
```

```
        dec = [to_str(p[to_arr(ch)]) for ch in chunks]              # substitution guess applied per column
        order_cur, _ = best_column_order(dec)
        seq = np.array([[to_arr(chunks[o])[r] for o in order_cur] for r in range(rows)]).ravel()   # cipher letters in plain order
        tri = seq[:-2] * 676 + seq[1:-1] * 26 + seq[2:]
        u, cnt = np.unique(tri, return_counts=True)
        ta, tb, tc = u // 676, (u // 26) % 26, u % 26
        sc = lambda q: float((cnt * LP3F[q[ta] * 676 + q[tb] * 26 + q[tc]]).sum())
        cur = sc(p)
        improved = True
        while improved:
            improved = False
            for i in range(25):
                for j in range(i + 1, 26):
                    p[i], p[j] = p[j], p[i]
                    s = sc(p)
                    if s > cur + 1e-9:
                        cur, improved = s, True
                    else:
                        p[i], p[j] = p[j], p[i]
    if cur > best[0]:
        plain_idx = p[seq]
        best = (cur, order_cur, to_str(plain_idx))
  return best[1], best[2]


# ------------------------------------------------------------------ insider "depth" attack (same key, many messages)
def depth_attack(cipher_msgs, kmax=9):
  """cipher_msgs: equal-length ciphertexts of the layered scheme  Vigenere -> columnar transposition,
     all produced with the SAME keys.  Step 1: every ciphertext position has one fixed Vigenere shift, so a
     Caesar frequency attack is run on that position across all messages.  Step 2: the remaining text is only
     transposed; the column order is found from bigram statistics summed over all messages."""
```

```
L = len(cipher_msgs[0])
M = np.array([to_arr(m) for m in cipher_msgs])                    # N x L
logE = np.log(ENG)
shifts = np.zeros(L, dtype=np.int64)
for j in range(L):
    cnt = np.bincount(M[:, j], minlength=26)
    shifts[j] = int(np.argmax([(cnt * logE[(np.arange(26) - k) % 26]).sum() for k in range(26)]))
T = (M - shifts[None, :]) % 26                                    # transposed plaintexts
best = None
for k in range(2, kmax + 1):
    if L % k:
        continue
    rows = L // k
    chunks = [T[:, j * rows:(j + 1) * rows] for j in range(k)]
    W = np.zeros((k, k))
    for a in range(k):
        for b in range(k):
            if a != b:
                W[a, b] = LP2[chunks[a], chunks[b]].sum()
    # Held-Karp on the aggregated matrix
    NEG = -1e18
    dp = [[NEG] * k for _ in range(1 << k)]
    par = [[-1] * k for _ in range(1 << k)]
    for i in range(k):
        dp[1 << i][i] = 0.0
    for mask in range(1, 1 << k):
        for last in range(k):
            v = dp[mask][last]
            if v == NEG or not (mask >> last) & 1:
                continue
            for nxt in range(k):
                if (mask >> nxt) & 1:
```

```
                continue
            nm = mask | (1 << nxt)
            s = v + W[last, nxt]
            if s > dp[nm][nxt]:
                dp[nm][nxt], par[nm][nxt] = s, last
    full = (1 << k) - 1
    last = max(range(k), key=lambda i: dp[full][i])
    total = dp[full][last]
    order, mask = [], full
    while last != -1:
        order.append(last)
        mask, last = mask ^ (1 << last), par[mask][last]
    order = order[::-1]
    if best is None or total / (k - 1) > best[0]:
        best = (total / (k - 1), k, order, chunks)
_, k, order, chunks = best
rows = L // k
plains = []
for i in range(len(cipher_msgs)):
    plains.append(to_str(np.array([chunks[o][i][r] for r in range(rows) for o in order])))
return plains, k, order, shifts
```

Appendix C – imgsec.py and labtable.py

imgsec.py

```
"""
imgsec.py - classical image encryption and steganography experiments (synthetic scan, no patient data).
"""
import io
import math
import random

import numpy as np
from PIL import Image
from scipy.stats import chi2



# ---------------------------------------------------------------- synthetic 256x256 "CT slice"
def phantom(size=256, seed=1, noise=2.5):
    """Head-like phantom: bone ring, brain tissue, ventricles, a bright lesion and scanner noise (8-bit)."""
    y, x = np.mgrid[0:size, 0:size].astype(float)
    cx = cy = size / 2

    def ell(a, b, x0=0, y0=0):
        return ((x - cx - x0) / a) ** 2 + ((y - cy - y0) / b) ** 2 <= 1

    img = np.full((size, size), 12.0)
    img[ell(100, 120)] = 215          # skull / bone
    img[ell(90, 110)] = 105           # brain tissue
    img[ell(18, 40, -22, -8)] = 55    # ventricle
    img[ell(18, 40, 22, -8)] = 55
    img[ell(14, 14, 38, 45)] = 165    # lesion
    img[ell(30, 12, 0, 70)] = 135     # a second structure
    rng = np.random.default_rng(seed)
    img += rng.normal(0, noise, img.shape)
```

```
    return np.clip(np.round(img), 0, 255).astype(np.uint8)


# ------------------------------------------------------------- two classical image ciphers
def shift_encrypt(img, k):
    """Intensity shift: C = (P + k) mod 256 (a Caesar cipher on pixel values)."""
    return ((img.astype(np.int16) + k) % 256).astype(np.uint8)


def shift_decrypt(img, k):
    return shift_encrypt(img, -k)


def perm_encrypt(img, key):
    """Pixel permutation (transposition of positions) with a key-seeded permutation."""
    p = np.random.default_rng(key).permutation(img.size)
    return img.reshape(-1)[p].reshape(img.shape)


def perm_decrypt(img, key):
    p = np.random.default_rng(key).permutation(img.size)
    out = np.empty(img.size, dtype=img.dtype)
    out[p] = img.reshape(-1)
    return out.reshape(img.shape)


def entropy(img):
    h = np.bincount(img.reshape(-1), minlength=256) / img.size
    h = h[h > 0]
    return float(-(h * np.log2(h)).sum())
```

```
def neighbour_correlation(img):
    a = img[:, :-1].astype(float).ravel()
    b = img[:, 1:].astype(float).ravel()
    return float(np.corrcoef(a, b)[0, 1])


def edge_energy(img):
    """Mean absolute horizontal gradient: high when structure (edges) is visible."""
    return float(np.abs(np.diff(img.astype(float), axis=1)).mean())


# ---------------------------------------------------------------- LSB steganography
def text_to_bits(s):
    return [(b >> (7 - i)) & 1 for b in s.encode("ascii") for i in range(8)]


def bits_to_text(bits):
    return bytes(int("".join(map(str, bits[i:i + 8])), 2) for i in range(0, len(bits) - len(bits) % 8, 8)).decode("ascii",
"replace")


def lsb_embed(img, message, key=None):
    """Embed a 16-bit length followed by the message bits into the LSBs (sequential, or in key-shuffled order)."""
    bits = [int(b) for b in format(len(message), "016b")] + text_to_bits(message)
    flat = img.reshape(-1).copy()
    if len(bits) > flat.size:
        raise ValueError("message does not fit in the image")
    order = np.arange(flat.size) if key is None else np.random.default_rng(key).permutation(flat.size)
    idx = order[:len(bits)]
    flat[idx] = (flat[idx] & 0xFE) | np.array(bits, dtype=np.uint8)
    return flat.reshape(img.shape)
```

```
def lsb_extract(img, key=None):
    flat = img.reshape(-1)
    order = np.arange(flat.size) if key is None else np.random.default_rng(key).permutation(flat.size)
    n = int("".join(str(int(b) & 1) for b in flat[order[:16]]), 2)
    if n * 8 + 16 > flat.size:
        return None
    bits = [int(flat[order[16 + i]]) & 1 for i in range(n * 8)]
    return bits_to_text(bits)


def lsb_embed_random_bits(img, n_bits, rng_seed=0, key=7):
    """Embed n_bits random bits (models an encrypted payload) in key-shuffled positions."""
    r = np.random.default_rng(rng_seed)
    flat = img.reshape(-1).copy()
    idx = np.random.default_rng(key).permutation(flat.size)[:n_bits]
    flat[idx] = (flat[idx] & 0xFE) | r.integers(0, 2, n_bits).astype(np.uint8)
    return flat.reshape(img.shape)


def psnr(a, b):
    mse = float(((a.astype(float) - b.astype(float)) ** 2).mean())
    return float("inf") if mse == 0 else 10 * math.log10(255 ** 2 / mse)


def chi_square_pairs(img):
    """Westfeld-Pfitzmann chi-square on pairs of values (2i, 2i+1).  Returns (statistic, p) where p is the
      probability that the pair counts are equalised, i.e. close to 1 means 'looks embedded'."""
    h = np.bincount(img.reshape(-1), minlength=256).astype(float)
    even, odd = h[0::2], h[1::2]
    exp = (even + odd) / 2
    m = exp > 5
```

```
    stat = float(((even[m] - exp[m]) ** 2 / exp[m]).sum())
    df = int(m.sum()) - 1
    return stat, float(1 - chi2.cdf(stat, df))


def jpeg_roundtrip(img, quality):
    buf = io.BytesIO()
    Image.fromarray(img).save(buf, format="JPEG", quality=quality)
    buf.seek(0)
    return np.array(Image.open(buf).convert("L"))


# ---------------------------------------------------------------- text steganography (word-gap spacing)
def text_embed(circular, message):
    """One space between two words = bit 0, two spaces = bit 1 (after a 16-bit length header)."""
    bits = [int(b) for b in format(len(message), "016b")] + text_to_bits(message)
    words = circular.split()
    if len(bits) > len(words) - 1:
        raise ValueError("circular is too short for this message")
    out = []
    for i, w in enumerate(words[:-1]):
        out.append(w + ("  " if i < len(bits) and bits[i] else " "))
    out.append(words[-1])
    return "".join(out)


def text_extract(stego):
    gaps = []
    i = 0
    while i < len(stego):
        if stego[i] == " ":
            j = i
```

```
        while j < len(stego) and stego[j] == " ":
            j += 1
        gaps.append(1 if j - i >= 2 else 0)
        i = j
    else:
        i += 1
if len(gaps) < 16:
    return None
n = int("".join(map(str, gaps[:16])), 2)
if 16 + 8 * n > len(gaps):
    return None
return bits_to_text(gaps[16:16 + 8 * n])
```

labtable.py

```
"""labtable.py - 24-row lab table protected by columnar transposition (18 columns) and classical checksums."""
import random
import classical as C

TESTS = ["GLUCOS", "HAEMOG", "CREATI", "SODIUM", "POTASS", "CHOLES", "BILIRU",
"UREAXX", "PLATEL", "WBCCNT"]
UNITS = {"GLUCOS": "MGDLX", "HAEMOG": "GDLXX", "CREATI": "MGDLX", "SODIUM":
"MMOLX", "POTASS": "MMOLX",
         "CHOLES": "MGDLX", "BILIRU": "MGDLX", "UREAXX": "MGDLX", "PLATEL": "KULXX",
"WBCCNT": "KULXX"}
W = 18      # characters per table row  (3 id + 6 test + 4 value + 5 unit)


def make_table(rng, rows=24):
    out = []
    for i in range(1, rows + 1):
        t = rng.choice(TESTS)
        out.append("R%02d%s%04d%s" % (i, t, rng.randint(10, 400), UNITS[t]))
```

```
    return out


def with_checksum(rows, kind):
    body = "".join(rows)
    if kind == "none":
        return body + "CHK" + "X" * 15
    f = C.checksum_sum if kind == "sum" else C.checksum_weighted
    return body + "CHK" + f(body) + "X" * 13


def verify(text, kind):
    body, last = text[:-W], text[-W:]
    if kind == "none":
        return True
    f = C.checksum_sum if kind == "sum" else C.checksum_weighted
    return last[:3] == "CHK" and last[3:5] == f(body)


def attack_cell_change(ct, k, rng):
    """No key: replace one random ciphertext symbol by a random symbol (changes exactly one plaintext cell)."""
    i = rng.randrange(len(ct))
    new = rng.choice([c for c in C.A36 if c != ct[i]])
    return ct[:i] + new + ct[i + 1:]


def attack_row_swap(ct, k, rng, rows):
    """No key: swap matrix rows r1, r2 inside every ciphertext column chunk -> two table rows are swapped."""
    r1, r2 = rng.sample(range(rows - 1), 2)       # never touches the checksum row
    out = list(ct)
    for c in range(k):
        a, b = c * rows + r1, c * rows + r2
```

```
        out[a], out[b] = out[b], out[a]
    return "".join(out)


def attack_cell_swap(ct, k, rng, rows):
    """No key: swap the symbols at one matrix row in two different column chunks -> two cells of a row are swapped."""
    r = rng.randrange(rows - 1)
    c1, c2 = rng.sample(range(k), 2)
    out = list(ct)
    a, b = c1 * rows + r, c2 * rows + r
    out[a], out[b] = out[b], out[a]
    return "".join(out)


def run(trials=2000, seed=5):
    rng = random.Random(seed)
    res = {}
    for kind in ("none", "sum", "weighted"):
        res[kind] = {}
        for name in ("cell change", "row swap", "cell swap in a row"):
            detected = changed = 0
            for _ in range(trials):
                tab = make_table(rng)
                plain = with_checksum(tab, kind)
                perm = list(range(W)); rng.shuffle(perm)
                ct = C.columnar_encrypt(plain, perm, C.A36)
                rows = len(plain) // W
                if name == "cell change":
                    bad = attack_cell_change(ct, W, rng)
                elif name == "row swap":
                    bad = attack_row_swap(ct, W, rng, rows)
```

```
            else:
                bad = attack_cell_swap(ct, W, rng, rows)
            dec = C.columnar_decrypt(bad, perm)
            if dec == plain:
                continue                # attack changed nothing (identical symbols) - not counted
            changed += 1
            if not verify(dec, kind):
                detected += 1
        res[kind][name] = {"changed": changed, "detected": detected, "rate": detected / max(changed, 1)}
return res
```

Appendix D – blp.py, notes.py and riskdata.py

blp.py

```
"""
blp.py - Bell-LaPadula engine with compartments, trusted subjects, declassification and break-glass.
Levels: PUBLIC < RESTRICTED < CONFIDENTIAL < SECRET < TOP SECRET.
A label is (level, set of compartments).  a dominates b <=> level(a) >= level(b) and comps(a) contains comps(b).
"""
LEVELS = {"PUBLIC": 0, "RESTRICTED": 1, "CONFIDENTIAL": 2, "SECRET": 3, "TOP SECRET": 4}



def dominates(a, b):
    return LEVELS[a[0]] >= LEVELS[b[0]] and set(a[1]) >= set(b[1])



SUBJECTS = {   # name: (label, trusted?)
    "Medical Director": (("TOP SECRET", {"GEN", "ONC", "PSY", "HIV"}), True),
    "Dr Rao (oncologist)": (("SECRET", {"GEN", "ONC"}), False),
    "Dr Iyer (psychiatrist)": (("SECRET", {"GEN", "PSY"}), False),
    "Nurse Meena": (("CONFIDENTIAL", {"GEN"}), False),
    "Lab tech Arun": (("CONFIDENTIAL", {"GEN"}), False),
    "Billing clerk": (("RESTRICTED", set()), False),
    "Patient": (("PUBLIC", set()), False),
}
OBJECTS = {
    "oncology diagnosis": ("SECRET", {"ONC"}),
    "psychiatric notes": ("SECRET", {"PSY"}),
    "HIV test result": ("SECRET", {"HIV"}),
    "allergy record": ("SECRET", {"GEN"}),
    "lab values": ("CONFIDENTIAL", {"GEN"}),
    "prescription": ("CONFIDENTIAL", {"GEN"}),
    "billing file": ("RESTRICTED", set()),
    "health leaflet": ("PUBLIC", set()),
```

```
    "executive summary": ("TOP SECRET", {"GEN", "ONC", "PSY", "HIV"}),
    "lab summary (declassified)": ("RESTRICTED", set()),
}


def decide(subject, action, obj, mode="normal"):
    """Return (decision, rule applied, explanation).  mode: 'normal', 'declassify' or 'break-glass'."""
    s_label, trusted = SUBJECTS[subject]
    o_label = OBJECTS[obj]
    if action == "read":
        if dominates(s_label, o_label):
            return "ALLOW", "simple security", f"{subject} {s_label[0]}/{sorted(s_label[1])} dominates {obj} {o_label[0]}/{sorted(o_label[1])}"
        why = "level too low" if LEVELS[s_label[0]] < LEVELS[o_label[0]] else f"missing compartment {sorted(set(o_label[1]) - set(s_label[1]))}"
        if mode == "break-glass":
            return "ALLOW (override)", "break-glass", f"BLP denies read up ({why}); emergency override with audit entry, time limit and review"
        return "DENY", "simple security (no read up)", f"{why}"
    if action == "write":
        if mode == "declassify" and trusted:
            return "ALLOW (trusted)", "declassification", f"trusted subject writes down to {obj} after sanitising the content; logged"
        if dominates(o_label, s_label):
            return "ALLOW", "*-property", f"{obj} dominates {subject}: write up or equal is permitted (no integrity check!)" if LEVELS[o_label[0]] < LEVELS[s_label[0]] or o_label != s_label and LEVELS[o_label[0]] > LEVELS[s_label[0]] else f"{obj} has the same label as {subject}"
        return "DENY", "*-property (no write down)", f"writing to {obj} would copy {s_label[0]} information to a lower label"
    raise ValueError("action must be read or write")
```

```
REQUESTS = [   # (subject, action, object, mode, note)
    ("Dr Rao (oncologist)", "read", "oncology diagnosis", "normal", ""),
    ("Nurse Meena", "read", "oncology diagnosis", "normal", ""),
    ("Nurse Meena", "read", "lab values", "normal", ""),
    ("Billing clerk", "read", "prescription", "normal", ""),
    ("Dr Iyer (psychiatrist)", "read", "oncology diagnosis", "normal", "same level, different compartment"),
    ("Dr Rao (oncologist)", "read", "HIV test result", "normal", "compartment not held"),
    ("Medical Director", "read", "psychiatric notes", "normal", ""),
    ("Lab tech Arun", "write", "lab values", "normal", ""),
    ("Lab tech Arun", "write", "billing file", "normal", "write down"),
    ("Billing clerk", "write", "lab values", "normal", "write up: allowed by BLP, but harmful"),
    ("Dr Rao (oncologist)", "write", "billing file", "normal", "doctor must record a billing note"),
    ("Medical Director", "write", "lab summary (declassified)", "declassify", "declassification by trusted subject"),
    ("Nurse Meena", "read", "allergy record", "break-glass", "emergency: Secret allergy record"),
    ("Patient", "read", "health leaflet", "normal", ""),
    ("Patient", "read", "prescription", "normal", "patient's own record"),
    ("Dr Rao (oncologist)", "write", "executive summary", "normal", "write up"),
]



def run_requests():
    rows = []
    for i, (s, a, o, mode, note) in enumerate(REQUESTS, 1):
        d, rule, why = decide(s, a, o, mode)
        rows.append({"n": i, "subject": s, "action": a, "object": o, "mode": mode, "decision": d, "rule": rule, "why": why, "note": note})
    return rows
```

notes.py

```
"""notes.py - synthetic clinical-note generator (no real patient data is used anywhere in this report)."""
import random
```

```
SYMPTOMS = ["persistent cough", "mild fever", "chest pain", "shortness of breath", "severe headache", "abdominal pain",
        "joint swelling", "dizziness and fatigue", "nausea and vomiting", "skin rash", "blurred vision", "back pain",
        "loss of appetite", "night sweats", "swelling of the legs", "difficulty sleeping"]
DIAGNOSES = ["type two diabetes", "hypertension", "community acquired pneumonia", "iron deficiency anaemia",
         "urinary tract infection", "bronchial asthma", "migraine", "hypothyroidism", "gastric ulcer",
         "acute bronchitis", "rheumatoid arthritis", "chronic kidney disease"]
DRUGS = ["AMOX", "METFORMIN", "AMLODIPINE", "PARACETAMOL", "IBUPROFEN", "OMEPRAZOLE", "SALBUTAMOL", "ATORVASTATIN"]
LABS = ["haemoglobin", "blood glucose", "serum creatinine", "white cell count", "platelet count", "serum sodium",
      "serum potassium", "cholesterol", "bilirubin", "urea"]
PLANS = ["a low salt diet and daily exercise", "rest and plenty of fluids", "a course of antibiotics for one week",
       "regular monitoring of blood pressure", "a repeat scan after two weeks", "referral to the specialist clinic",
       "a review of the current medication", "an ultrasound examination of the abdomen"]
TEMPLATES = [
   "The patient presents with {s1} for {n} days and also reports {s2}.",
   "On examination the patient was alert and oriented and the vital signs were stable.",
   "Blood pressure was {bp} and the heart rate was {hr} beats per minute.",
   "Laboratory results show that the {l1} is within the normal range while the {l2} is slightly raised.",
   "We advise {p} and a review in the clinic after {n} days.",
   "The specialist recommends {p} together with further monitoring of the {l1}.",
   "There is no history of allergy to any medicine and the family history is not significant.",
   "The patient has been informed about the treatment and understands the need for regular follow up.",
   "The nurse recorded the temperature and the weight of the patient at the time of the visit.",
   "The pain started suddenly and has been getting worse over the last few days.",
   "The results of the tests were discussed with the patient and the relatives in the ward.",
   "A detailed report will be sent to the central hospital for the opinion of the specialist.",
]
```

```
def note(rng):
    """One clinical note: predictable header lines followed by a few narrative sentences."""
    pid = "PT-%05d" % rng.randint(10000, 99999)
    head = "PATIENT ID: %s. DIAGNOSIS: %s. PRESCRIPTION: %s %dMG. " % (
        pid, rng.choice(DIAGNOSES), rng.choice(DRUGS), rng.choice([250, 500, 750]))
    body = []
    for _ in range(rng.randint(4, 8)):
        t = rng.choice(TEMPLATES)
        body.append(t.format(s1=rng.choice(SYMPTOMS), s2=rng.choice(SYMPTOMS), n=rng.randint(2, 14),
                             bp="%d over %d" % (rng.randint(100, 160), rng.randint(60, 100)), hr=rng.randint(55, 110),
                             l1=rng.choice(LABS), l2=rng.choice(LABS), p=rng.choice(PLANS)))
    return head + " ".join(body)


def corpus(seed, letters, header=True):
    """Concatenated notes (letters only) of at least `letters` characters."""
    rng = random.Random(seed)
    out, total = [], 0
    while total < letters:
        n = note(rng)
        t = "".join(c for c in n.upper() if "A" <= c <= "Z")
        out.append(t)
        total += len(t)
    return "".join(out)
```

riskdata.py

```
"""riskdata.py - risk register used by the report (likelihood and impact scored 1-5)."""
IMPACT_SCALE = {1: "negligible", 2: "minor privacy leak, no care effect", 3: "serious privacy breach of individual patients",
                4: "delayed care or large privacy breach", 5: "risk to life or bulk exposure of the whole network"}
```

```
ATTACKS = [   # code, name, type, likelihood, impact, rationale
   ("A1", "Eavesdropping on the public network", "passive", 5, 3, "Anyone on the path can copy traffic; harm is privacy (diagnoses), not direct physical harm."),
   ("A2", "Traffic analysis", "passive", 4, 2, "Message sizes and timing reveal which clinic sends which kind of record, but not the content."),
   ("A3", "Modification of prescription / dosage", "active", 4, 5, "No integrity protection: an altered dose is accepted as valid; wrong dose can kill."),
   ("A4", "Masquerade as a doctor", "active", 3, 5, "Shared keys give no proof of origin; a forged order or record request is accepted."),
   ("A5", "Replay of old orders", "active", 4, 4, "A captured valid order can be sent again (duplicate dose, repeated test, cancelled order revived)."),
   ("A6", "Denial of service on the records system", "active", 3, 4, "Care is delayed, but data is not changed or disclosed."),
]
AFTER = [   # re-assessment after the insider incident (Task 12)
   ("A7", "Insider copies ciphertexts and recovers the key", "passive then active", 5, 5, "Hundreds of known-header messages under one key: key recovered, every message readable and forgeable."),
   ("A3", "Modification of prescription / dosage", "active", 5, 5, "Insider with the key can alter any message and re-encrypt it; no check can stop this."),
   ("A4", "Masquerade as a doctor", "active", 5, 5, "The key is shared by all 25 clinics, so a clinic insider can impersonate any other site."),
   ("A1", "Eavesdropping on the public network", "passive", 5, 5, "All past and future traffic of all clinics becomes readable."),
   ("A5", "Replay of old orders", "active", 5, 4, "Insider can replay and also craft fresh valid orders."),
   ("A6", "Denial of service on the records system", "active", 3, 4, "Unchanged by the key compromise."),
   ("A2", "Traffic analysis", "passive", 4, 2, "Unchanged."),
]
def ranked(rows):
   return sorted(rows, key=lambda r: (-(r[3] * r[4]), -r[4], r[0]))
```

Appendix E – run_all.py and worked.py

run_all.py

```
"""run_all.py - runs every experiment of the report and writes results.json.  Usage:  python run_all.py"""
import json, math, random, time
import numpy as np
import classical as C
import attacks as A
import notes
import imgsec as I
import labtable as L
import blp

SEED = 2026
rng = random.Random(SEED)
R = {}
LOG = []
def log(*a):
    s = " ".join(str(x) for x in a); print(s); LOG.append(s)

# ---------------------------------------------------------------- plaintexts and keys
BASE = notes.corpus(31337, 100000)[:100000]          # identical plaintext for all schemes (letters only)
POOL = notes.corpus(2024, 120000)                    # windows for attack trials
KEYS = {"caesar": 11, "mono": C.random_mono_key(random.Random(1)), "vig": "MEDICAL", "vig12":
"HOSPITALCARE",
        "col": [4, 2, 6, 0, 3, 5, 1], "hill": [[1, 1, 3], [1, 2, 1], [2, 8, 1]]}
R["keys"] = {k: v for k, v in KEYS.items()}
log("plaintext IC (100000 letters):", round(C.ic(BASE), 4))

# ---------------------------------------------------------------- 1. key spaces
fact = math.factorial
ks = {
    "Caesar": 25, "Monoalphabetic": fact(26), "Vigenere m=7": 26 ** 7, "Vigenere m=12": 26 ** 12,
```

```
    "Columnar 7 cols": fact(7), "Columnar 10 cols": fact(10), "Columnar 18 cols": fact(18),
    "Hill 3x3 mod 26": C.count_invertible(3, 26), "Hill 3x3 mod 36": C.count_invertible(3, 36),
    "Mono + columnar(7)": fact(26) * fact(7), "One-time pad, 336 letters": 26 ** 336,
}
R["keyspace"] = {k: {"exact": str(v) if v < 10 ** 30 else None, "log2": math.log2(v), "log10": math.log10(v)}
for k, v in ks.items()}
R["hill_fraction36"] = C.count_invertible(3, 36) / 36 ** 9
R["hill_fraction26"] = C.count_invertible(3, 26) / 26 ** 9
for k, v in R["keyspace"].items(): log("keyspace", k, "log2 =", round(v["log2"], 1))

# ---------------------------------------------------------------- 2. encryption of notes, round trip, IC, histograms
def enc_all(pt):
    out = {"caesar": C.caesar_encrypt(pt, KEYS["caesar"]), "mono": C.mono_encrypt(pt, KEYS["mono"]),
           "vig": C.vigenere_encrypt(pt, KEYS["vig"]), "col": C.columnar_encrypt(pt, KEYS["col"])}
    out["hill"], padn = C.hill_encrypt(pt, KEYS["hill"])
    out["product"] = C.columnar_encrypt(out["mono"], KEYS["col"])
    return out, padn
def dec_all(ct, n, padn):
    return {"caesar": C.caesar_decrypt(ct["caesar"], KEYS["caesar"]), "mono": C.mono_decrypt(ct["mono"],
KEYS["mono"]),
            "vig": C.vigenere_decrypt(ct["vig"], KEYS["vig"]), "col": C.columnar_decrypt(ct["col"], KEYS["col"],
n),
            "hill": C.hill_decrypt(ct["hill"], KEYS["hill"], padn),
            "product": C.mono_decrypt(C.columnar_decrypt(ct["product"], KEYS["col"], n), KEYS["mono"])}
R["roundtrip"] = {}
for n in (200, 1000, 10000):
    pt = BASE[:n]; ct, padn = enc_all(pt); dec = dec_all(ct, n, padn)
    R["roundtrip"][n] = {k: dec[k] == pt for k in dec}
    log("round trip", n, R["roundtrip"][n])
ex200, padn = enc_all(BASE[:200])
R["example200"] = {"plain": BASE[:60], **{k: v[:60] for k, v in ex200.items()}}
P10, _ = enc_all(BASE[:10000])
```

```
R["ic10k"] = {"plaintext": C.ic(BASE[:10000]), **{k: C.ic(v, C.A36 if k == "hill" else C.A26) for k, v in P10.items()}}
R["ic10k"]["random26"] = 1 / 26; R["ic10k"]["random36"] = 1 / 36
log("IC 10k", {k: round(v, 4) for k, v in R["ic10k"].items()})
R["freq10k"] = {"plaintext": C.freq(BASE[:10000]), **{k: C.freq(v, C.A36 if k == "hill" else C.A26) for k, v in P10.items()}}
R["top_letters_plain"] = sorted(zip(C.A26, C.freq(BASE[:10000])), key=lambda x: -x[1])[:5]
R["top_letters_mono"] = sorted(zip(C.A26, C.freq(P10["mono"])), key=lambda x: -x[1])[:5]

# ---------------------------------------------------------------- 3. worked breaks at n = 1008 (one example each)
n = 1008
pt = POOL[5000:5000 + n]
w = {}
ct = C.caesar_encrypt(pt, 11); k, dec = A.caesar_break(ct); w["caesar"] = {"key": k, "acc": A.accuracy(dec, pt)}
ct = C.mono_encrypt(pt, KEYS["mono"]); key_c2p, dec = A.mono_break(ct, random.Random(3))
w["mono"] = {"true_key": KEYS["mono"], "found_cipher_to_plain": key_c2p, "acc": A.accuracy(dec, pt)}
ct = C.vigenere_encrypt(pt, KEYS["vig"]); nd, votes = A.kasiski(ct); key, dec, m = A.vigenere_break(ct)
ics = A.avg_ic_by_length(ct, 21)
w["vig"] = {"key": key, "m": m, "acc": A.accuracy(dec, pt), "repeat_distances": nd, "kasiski_top": votes.most_common(5), "ic_by_m": ics}
perm = KEYS["col"]; ct = C.columnar_encrypt(pt, perm); k, order, dec = A.transposition_break(ct)
w["col"] = {"k": k, "order": order, "acc": A.accuracy(dec, pt)}
R["worked_breaks"] = w
log("worked breaks", {k: v.get("acc") for k, v in w.items()}, "kasiski", w["vig"]["kasiski_top"], "key", w["vig"]["key"])

# ---------------------------------------------------------------- 4. attack success versus ciphertext length
LENS = [56, 112, 224, 448, 896, 1792, 3584, 7168]
TR = {"caesar": 100, "mono": 30, "monofreq": 30, "vig": 100, "col": 100, "product": 12}
succ = {k: [] for k in TR}; accm = {k: [] for k in TR}
r2 = random.Random(77)
def window(n):
```

```
    s = r2.randrange(0, len(POOL) - n); return POOL[s:s + n]
t0 = time.time()
for n in LENS:
    res = {k: [] for k in TR}
    for _ in range(TR["caesar"]):
        pt = window(n); k0 = r2.randrange(1, 26); res["caesar"].append(A.accuracy(A.caesar_break(C.caesar_encrypt(pt, k0))[1], pt))
    for _ in range(TR["vig"]):
        pt = window(n); key = "".join(r2.choice(C.A26) for _ in range(7)); res["vig"].append(A.accuracy(A.vigenere_break(C.vigenere_encrypt(pt, key))[1], pt))
    for _ in range(TR["col"]):
        pt = window(n); pm = list(range(7)); r2.shuffle(pm)
        res["col"].append(A.accuracy(A.transposition_break(C.columnar_encrypt(pt, pm))[2], pt))
    for _ in range(TR["mono"]):
        pt = window(n); key = C.random_mono_key(r2); ct = C.mono_encrypt(pt, key)
        res["mono"].append(A.accuracy(A.mono_break(ct, r2)[1], pt))
        # frequency-rank guess only (no n-gram refinement)
        cr = np.argsort(-np.bincount(A.to_arr(ct), minlength=26)); pr = np.argsort(-A.LP1); mp = np.zeros(26, dtype=int); mp[cr] = pr
        res["monofreq"].append(A.accuracy(A.to_str(mp[A.to_arr(ct)]), pt))
    for _ in range(TR["product"]):
        pt = window(n); pm = list(range(7)); r2.shuffle(pm); key = C.random_mono_key(r2)
        ct = C.columnar_encrypt(C.mono_encrypt(pt, key), pm)
        res["product"].append(A.accuracy(A.product_break(ct, 7, r2)[1], pt))
    for k in TR:
        succ[k].append(float(np.mean([a >= 0.9 for a in res[k]]))); accm[k].append(float(np.mean(res[k])))
    log("length", n, {k: round(succ[k][-1], 2) for k in TR}, round(time.time() - t0), "s")
R["success"] = {"lengths": LENS, "trials": TR, "rate": succ, "mean_accuracy": accm}

# product cipher: wrong attack (mono hill climb ignoring the transposition)
n = 1792; vals = []
for _ in range(10):
```

```
    pt = window(n); pm = list(range(7)); r2.shuffle(pm); key = C.random_mono_key(r2)
    ct = C.columnar_encrypt(C.mono_encrypt(pt, key), pm)
    vals.append(A.accuracy(A.mono_break(ct, r2)[1], pt))
R["mono_attack_on_product_1792"] = float(np.mean(vals))
log("mono attack on product (n=1792) mean accuracy", round(R["mono_attack_on_product_1792"], 3))


# ---------------------------------------------------------------- 5. Hill: IDs, known-plaintext attack
K = KEYS["hill"]
h = {"det": C.det(K), "det_mod36": C.det(K, 36), "det_inv": pow(C.det(K, 36), -1, 36), "adj": C.adjugate(K, 36), "inv": C.mat_inv(K, 36)}
h["check"] = C.mat_mul(K, h["inv"], 36)
bad = [[6, 24, 1], [13, 16, 10], [20, 17, 15]]
h["classic_key"] = {"det": C.det(bad), "mod26": C.det(bad, 26), "mod36": C.det(bad, 36), "gcd26": math.gcd(C.det(bad, 26), 26), "gcd36": math.gcd(C.det(bad, 36), 36)}
cases = ["PT-20481", "AMOX 500MG", "PT-99317 IBUPROFEN 750MG", "A", "", "9", "PT-20481, AMOX 500MG!"]
h["cases"] = []
for t in cases:
    try:
        c, pd = C.hill_encrypt(t, K); d = C.hill_decrypt(c, K, pd)
        h["cases"].append({"input": t, "normalised": C.normalize(t, C.A36), "cipher": c, "pad": pd, "decrypted": d, "ok": d == C.normalize(t, C.A36)})
    except ValueError as e:
        h["cases"].append({"input": t, "error": str(e)})
# known-plaintext attack from predictable headers
header = "PATIENTIDPTDIAGNOSIS"
hk_c, hk_pad = C.hill_encrypt(header + "PT20481", K)
pairs = [(header[i:i + 3], hk_c[i:i + 3]) for i in range(0, 18, 3)]
Krec, combo = C.hill_kpa(pairs, 3)
h["kpa"] = {"header": header, "blocks": pairs, "recovered": Krec, "match": Krec == [[x % 36 for x in r] for r in K], "combo": combo}
# how many random known blocks are needed?
```

```
tri = random.Random(8); cnt = 0; trials = 3000
for _ in range(trials):
    blocks = ["".join(tri.choice(C.A36) for _ in range(3)) for _ in range(3)]
    P = [[C.A36.index(b[r]) for b in blocks] for r in range(3)]
    cnt += math.gcd(C.det(P, 36), 36) == 1
h["kpa_three_random_blocks_ok"] = cnt / trials
need = []
for _ in range(500):
    blocks = []
    while True:
        blocks.append("".join(tri.choice(C.A36) for _ in range(3)))
        if len(blocks) >= 3:
            ok = any(math.gcd(C.det([[C.A36.index(blocks[b][r]) for b in cmb] for r in range(3)], 36), 36) == 1 for
cmb in __import__("itertools").combinations(range(len(blocks)), 3))
            if ok: break
    need.append(len(blocks))
h["kpa_blocks_needed_mean"] = float(np.mean(need)); h["kpa_blocks_needed_max"] = max(need)
R["hill"] = h
log("hill KPA match", h["kpa"]["match"], "P(3 random blocks invertible)", h["kpa_three_random_blocks_ok"],
"blocks needed mean", h["kpa_blocks_needed_mean"])


# ---------------------------------------------------------------- 6. timing
def timeit(f, reps):
    t = time.perf_counter()
    for _ in range(reps): f()
    return (time.perf_counter() - t) / reps
SIZES = [100, 1000, 10000, 100000]
tm = {k: {"enc": [], "dec": []} for k in ("Caesar", "Monoalphabetic", "Vigenere", "Columnar", "Hill (mod 36)",
"Mono + columnar")}
for n in SIZES:
    pt = BASE[:n]; reps = 20 if n <= 1000 else (5 if n <= 10000 else 2)
```

```
    mono_c = C.mono_encrypt(pt, KEYS["mono"]); vc = C.vigenere_encrypt(pt, KEYS["vig"]); cc = C.columnar_encrypt(pt, KEYS["col"]); hc, hp = C.hill_encrypt(pt, KEYS["hill"])
    pc = C.columnar_encrypt(mono_c, KEYS["col"]); cac = C.caesar_encrypt(pt, 11)
    tm["Caesar"]["enc"].append(timeit(lambda: C.caesar_encrypt(pt, 11), reps)); tm["Caesar"]["dec"].append(timeit(lambda: C.caesar_decrypt(cac, 11), reps))
    tm["Monoalphabetic"]["enc"].append(timeit(lambda: C.mono_encrypt(pt, KEYS["mono"]), reps)); tm["Monoalphabetic"]["dec"].append(timeit(lambda: C.mono_decrypt(mono_c, KEYS["mono"]), reps))
    tm["Vigenere"]["enc"].append(timeit(lambda: C.vigenere_encrypt(pt, KEYS["vig"]), reps)); tm["Vigenere"]["dec"].append(timeit(lambda: C.vigenere_decrypt(vc, KEYS["vig"]), reps))
    tm["Columnar"]["enc"].append(timeit(lambda: C.columnar_encrypt(pt, KEYS["col"]), reps)); tm["Columnar"]["dec"].append(timeit(lambda: C.columnar_decrypt(cc, KEYS["col"], n), reps))
    tm["Hill (mod 36)"]["enc"].append(timeit(lambda: C.hill_encrypt(pt, KEYS["hill"]), reps)); tm["Hill (mod 36)"]["dec"].append(timeit(lambda: C.hill_decrypt(hc, KEYS["hill"], hp), reps))
    tm["Mono + columnar"]["enc"].append(timeit(lambda: C.columnar_encrypt(C.mono_encrypt(pt, KEYS["mono"]), KEYS["col"]), reps))
    tm["Mono + columnar"]["dec"].append(timeit(lambda: C.mono_decrypt(C.columnar_decrypt(pc, KEYS["col"], n), KEYS["mono"]), reps))
R["timing"] = {"sizes": SIZES, "ms": {k: {"enc": [x * 1000 for x in v["enc"]], "dec": [x * 1000 for x in v["dec"]]} for k, v in tm.items()}}
log("timing 100000 enc ms", {k: round(v["enc"][-1] * 1000, 2) for k, v in tm.items()})

# error propagation: one wrong ciphertext symbol -> how many plaintext letters wrong
ep = {}
pt = BASE[:600]; t2 = random.Random(4)
def wrong(a, b): return sum(1 for x, y in zip(a, b) if x != y) + abs(len(a) - len(b))
for name, enc, dec, alpha in [("Caesar", lambda p: C.caesar_encrypt(p, 11), lambda c: C.caesar_decrypt(c, 11), C.A26),
                              ("Monoalphabetic", lambda p: C.mono_encrypt(p, KEYS["mono"]), lambda c: C.mono_decrypt(c, KEYS["mono"]), C.A26),
                              ("Vigenere", lambda p: C.vigenere_encrypt(p, KEYS["vig"]), lambda c: C.vigenere_decrypt(c, KEYS["vig"]), C.A26),
```

```
                ("Columnar", lambda p: C.columnar_encrypt(p, KEYS["col"]), lambda c: C.columnar_decrypt(c, KEYS["col"], 600), C.A26)]:
    c = enc(pt); tot = 0
    for _ in range(200):
        i = t2.randrange(len(c)); bad = c[:i] + t2.choice([x for x in alpha if x != c[i]]) + c[i + 1:]
        tot += wrong(dec(bad), pt)
    ep[name] = tot / 200
c, pd = C.hill_encrypt(pt, KEYS["hill"]); tot = 0
for _ in range(200):
    i = t2.randrange(len(c)); bad = c[:i] + t2.choice([x for x in C.A36 if x != c[i]]) + c[i + 1:]
    tot += wrong(C.hill_decrypt(bad, KEYS["hill"], pd), pt)
ep["Hill (mod 36)"] = tot / 200
R["error_propagation"] = ep
log("error propagation", ep)


# ---------------------------------------------------------------- 7. malleability / integrity demonstrations
m = {}
vk = "KEY7"
plain = "AMOX500MG"; c = C.vigenere_encrypt(plain, vk, C.A36)
pos = 4; delta = 4        # '5' -> '9' at the first digit
cv = C.A36.index(c[pos]); forged = c[:pos] + C.A36[(cv + delta) % 36] + c[pos + 1:]
m["vig36"] = {"plain": plain, "key": vk, "cipher": c, "forged": forged, "decrypted_forged": C.vigenere_decrypt(forged, vk, C.A36)}
log("malleability", m["vig36"])
R["malleability"] = m
R["labtable"] = L.run(2000)
log("lab table", {k: {a: round(b["rate"], 3) for a, b in v.items()} for k, v in R["labtable"].items()})
# insider with the key re-computes the checksum
rl = random.Random(12); tab = L.make_table(rl); perm = list(range(18)); rl.shuffle(perm)
plain = L.with_checksum(tab, "weighted"); ctx = C.columnar_encrypt(plain, perm, C.A36)
dec = C.columnar_decrypt(ctx, perm)
row = 5; bad_tab = tab[:]; bad_tab[row] = bad_tab[row][:9] + "9999" + bad_tab[row][13:]
```

```
forged_plain = L.with_checksum(bad_tab, "weighted"); forged_ct = C.columnar_encrypt(forged_plain, perm, C.A36)
R["insider_forgery"] = {"row_before": tab[row], "row_after": bad_tab[row], "accepted": L.verify(C.columnar_decrypt(forged_ct, perm), "weighted")}
log("insider forgery accepted:", R["insider_forgery"])

# ------------------------------------------------------------------ 8. images and steganography
img = I.phantom()
e1 = I.shift_encrypt(img, 77); e2 = I.perm_encrypt(img, 123); e3 = I.perm_encrypt(I.shift_encrypt(img, 77), 123)
st = lambda x: {"entropy": I.entropy(x), "corr": I.neighbour_correlation(x), "edge": I.edge_energy(x)}
R["image"] = {"orig": st(img), "shift": st(e1), "perm": st(e2), "both": st(e3),
              "ok_shift": bool(np.array_equal(I.shift_decrypt(e1, 77), img)), "ok_perm": bool(np.array_equal(I.perm_decrypt(e2, 123), img)),
              "ok_both": bool(np.array_equal(I.shift_decrypt(I.perm_decrypt(e3, 123), 77), img))}
log("image", {k: (v if not isinstance(v, dict) else {a: round(b, 3) for a, b in v.items()}) for k, v in R["image"].items()})
report = "PT-20481 ALLERGY: PENICILLIN. RX AMOX 500MG BD FOR 7 DAYS. REVIEW IN 2 WEEKS."
s_seq = I.lsb_embed(img, report); s_key = I.lsb_embed(img, report, key=99)
R["stego"] = {"message": report, "bits": len(report) * 8 + 16, "capacity_bits": img.size, "psnr_seq": I.psnr(img, s_seq), "psnr_key": I.psnr(img, s_key),
              "max_diff": int(np.abs(s_seq.astype(int) - img.astype(int)).max()), "changed_pixels": int((s_seq != img).sum()),
              "extract_ok": I.lsb_extract(s_seq) == report and I.lsb_extract(s_key, key=99) == report}
lesion = (slice(110, 140), slice(150, 180))
def cnr(a):
    les = a[128 - 8:128 + 8, 128 + 30:128 + 46].astype(float); bg = a[128 - 8:128 + 8, 128 - 8:128 + 8].astype(float)
    return float(abs(les.mean() - bg.mean()) / bg.std())
R["stego"]["cnr_before"] = cnr(img); R["stego"]["cnr_after_100pct"] = cnr(I.lsb_embed_random_bits(img, img.size))
rates = [0.0, 0.01, 0.05, 0.1, 0.25, 0.5, 0.75, 1.0]
```

```
rr = {"rates": rates, "psnr": [], "chi_p": [], "chi_stat": []}
for rate in rates:
    sx = img if rate == 0 else I.lsb_embed_random_bits(img, int(rate * img.size))
    rr["psnr"].append(None if rate == 0 else I.psnr(img, sx)); stat, p = I.chi_square_pairs(sx); rr["chi_stat"].append(stat); rr["chi_p"].append(p)
R["stego"]["rates"] = rr
# sequential embedding detected by the chi-square test on the first window (attacker knows nothing but tests windows)
seq = {"frac": [], "p": []}
for rate in (0.05, 0.1, 0.25, 0.5, 1.0):
    n_bits = int(rate * img.size); flat = img.reshape(-1).copy(); rb = np.random.default_rng(1).integers(0, 2, n_bits).astype(np.uint8)
    flat[:n_bits] = (flat[:n_bits] & 0xFE) | rb; win = flat[:n_bits]
    h = np.bincount(win, minlength=256).astype(float); ev, od = h[0::2], h[1::2]; ex = (ev + od) / 2; mk = ex > 5
    stat = float(((ev[mk] - ex[mk]) ** 2 / ex[mk]).sum()); from scipy.stats import chi2; p = float(1 - chi2.cdf(stat, int(mk.sum()) - 1))
    seq["frac"].append(rate); seq["p"].append(p)
R["stego"]["sequential_window"] = seq
jp = {"quality": [95, 90, 75, 50], "ber": [], "recovered": []}
for q in jp["quality"]:
    j = I.jpeg_roundtrip(s_seq, q); bits_a = [int(b) & 1 for b in s_seq.reshape(-1)[:R["stego"]["bits"]]]; bits_b = [int(b) & 1 for b in j.reshape(-1)[:R["stego"]["bits"]]]
    jp["ber"].append(float(np.mean([a != b for a, b in zip(bits_a, bits_b)]))); jp["recovered"].append(I.lsb_extract(j) == report)
R["stego"]["jpeg"] = jp
log("stego", {k: v for k, v in R["stego"].items() if k not in ("rates", "message", "sequential_window", "jpeg")}, jp)
# text steganography
circular = " ".join(notes.note(random.Random(21)) for _ in range(4)) + " " + " ".join(notes.note(random.Random(22)) for _ in range(2))
tmsg = "PT20481 ALLERGY PENICILLIN"
ts = I.text_embed(circular, tmsg)
```

```
R["textstego"] = {"words": len(circular.split()), "capacity_bits": len(circular.split()) - 1 - 16, "capacity_chars": (len(circular.split()) - 17) // 8, "message": tmsg,
                  "extract_ok": I.text_extract(ts) == tmsg, "double_spaces": ts.count("  "), "after_whitespace_collapse": I.text_extract(" ".join(ts.split())),
                  "after_retype_single_space": I.text_extract(ts.replace("  ", " "))}
log("text stego", R["textstego"])
# encrypt-then-embed: Vigenere-36 encrypted report (ASCII bits) at the same payload
enc_report = C.vigenere_encrypt(report, "RURALKEY", C.A36)
R["encrypt_then_embed"] = {"cipher": enc_report, "extract_ok": I.lsb_extract(I.lsb_embed(img, enc_report)) == enc_report,
                           "decrypt_ok": C.vigenere_decrypt(enc_report, "RURALKEY", C.A36) == C.normalize(report, C.A36)}

# ---------------------------------------------------------------- 9. insider depth attack
LMSG = 336
def msgs(N, seed):
    rr_ = random.Random(seed); out = []
    while len(out) < N:
        t = "".join(c for c in notes.note(rr_).upper() if "A" <= c <= "Z")
        if len(t) >= LMSG: out.append(t[:LMSG])
    return out
NS = [1, 2, 5, 10, 15, 20, 30, 50, 100, 300]
dep = {"N": NS, "same_key": [], "otp": None}
r3 = random.Random(31)
for N in NS:
    accs = []
    for trial in range(6):
        vkey = "".join(r3.choice(C.A26) for _ in range(12)); pm = list(range(7)); r3.shuffle(pm)
        pts = msgs(max(N, 2), 1000 + 7 * N + trial); cts = [C.columnar_encrypt(C.vigenere_encrypt(p, vkey), pm) for p in pts]
        if N == 1:
            cts, pts = cts[:1], pts[:1]
```

```
        dec, k, order, sh = A.depth_attack(cts if N > 1 else cts * 1)
        accs.append(float(np.mean([A.accuracy(d, p) for d, p in zip(dec, pts)])))
    dep["same_key"].append(float(np.mean(accs)))
accs = []
for trial in range(6):
    pm = list(range(7)); r3.shuffle(pm); pts = msgs(300, 5000 + trial)
    cts = [C.columnar_encrypt(C.vigenere_encrypt(p, "".join(r3.choice(C.A26) for _ in range(LMSG))), pm) for p in pts]
    dec, k, order, sh = A.depth_attack(cts); accs.append(float(np.mean([A.accuracy(d, p) for d, p in zip(dec, pts)])))
dep["otp"] = float(np.mean(accs))
# one-message known-header attack on Vigenere (12 letters key) and column-position variance
vkey = "HOSPITALCARE"; pts = msgs(3, 55); cts = [C.vigenere_encrypt(p, vkey) for p in pts]
hdr = "PATIENTIDPTDIAGNOSIS"
recovered = "".join(C.A26[(C.A26.index(cts[0][i]) - C.A26.index(hdr[i])) % 26] for i in range(12))
dep["header_kpa"] = {"cipher_first20": cts[0][:20], "header": hdr, "keystream12": recovered, "true_key": vkey, "match": recovered == vkey}
R["depth"] = dep
log("depth", dep)
# ---------------------------------------------------------------- 10. key management numbers
R["keymgmt"] = {"pairs_full_mesh": 26 * 25 // 2, "star": 25, "notes_per_day": 100, "chars_per_note": 1000,
                "otp_bytes_per_clinic_year": 100 * 1000 * 365, "otp_total_mb_year": 100 * 1000 * 365 * 25 / 1e6}
# ---------------------------------------------------------------- 11. Bell-LaPadula
R["blp"] = blp.run_requests()
for r in R["blp"]: log("BLP", r["n"], r["subject"], r["action"], r["object"], "->", r["decision"])
json.dump(R, open("results.json", "w"), indent=1, default=str)
open("run_log.txt", "w").write("\n".join(LOG))
print("done")
```

worked.py

```
"""worked.py - step-by-step worked examples (message of 20+ letters from the scenario) written to worked.json."""
```

```
import json, time
import classical as C
import notes
import riskdata as RK

MSG = "SEND LAB REPORT FOR PATIENT NOW"
P = C.normalize(MSG)
W = {"message": MSG, "normalised": P, "length": len(P)}
# Caesar
k = 11
W["caesar"] = {"k": k, "cipher": C.caesar_encrypt(P, k), "plain_back": C.caesar_decrypt(C.caesar_encrypt(P, k), k),
               "rows": [[c, C.A26.index(c), (C.A26.index(c) + k) % 26, C.caesar_encrypt(c, k)] for c in P[:12]]}
# Monoalphabetic keyword cipher
key = C.keyword_key("HEALTHCARE")
W["mono"] = {"keyword": "HEALTHCARE", "alphabet": key, "cipher": C.mono_encrypt(P, key), "plain_back": C.mono_decrypt(C.mono_encrypt(P, key), key)}
# Vigenere
vk = "MEDICAL"
rows = []
for i, ch in enumerate(P):
    kk = vk[i % len(vk)]
    rows.append([i + 1, ch, C.A26.index(ch), kk, C.A26.index(kk), (C.A26.index(ch) + C.A26.index(kk)) % 26, C.A26[(C.A26.index(ch) + C.A26.index(kk)) % 26]])
W["vig"] = {"key": vk, "rows": rows, "cipher": C.vigenere_encrypt(P, vk), "plain_back": C.vigenere_decrypt(C.vigenere_encrypt(P, vk), vk)}
# Columnar
perm = [4, 2, 6, 0, 3, 5, 1]
padded = C.pad_to(P, 7)
grid = [padded[i:i + 7] for i in range(0, len(padded), 7)]
W["col"] = {"perm": perm, "grid": grid, "cipher": C.columnar_encrypt(P, perm), "plain_back": C.columnar_decrypt(C.columnar_encrypt(P, perm), perm, len(P)),
```

```
        "read_order": [f"column {c + 1}" for c in perm], "columns": {str(c + 1): padded[c::7] for c in range(7)}}
# Hill
K = [[1, 1, 3], [1, 2, 1], [2, 8, 1]]
text = "PT-20481"; norm = C.normalize(text, C.A36); c, padn = C.hill_encrypt(text, K)
blocks = []
for b in range(0, len(norm) + padn, 3):
    blk = (norm + "X" * padn)[b:b + 3]; vec = [C.A36.index(x) for x in blk]
    prod = [sum(K[i][j] * vec[j] for j in range(3)) for i in range(3)]
    blocks.append({"block": blk, "vec": vec, "sums": prod, "mod": [x % 36 for x in prod], "cipher":
"".join(C.A36[x % 36] for x in prod)})
W["hill"] = {"input": text, "normalised": norm, "pad": padn, "blocks": blocks, "cipher": c, "plain_back":
C.hill_decrypt(c, K, padn)}
# dosage example
c2, p2 = C.hill_encrypt("AMOX 500MG", K)
W["hill_dose"] = {"input": "AMOX 500MG", "cipher": c2, "pad": p2, "plain_back": C.hill_decrypt(c2, K, p2)}
# size tests with identical plaintexts
base = notes.corpus(31337, 100000)[:100000]
KEYS = {"mono": key, "vig": "MEDICAL", "col": perm}
sizes = {}
for n in (100, 1000, 10000, 100000):
    pt = base[:n]; r = {}
    r["Caesar"] = C.caesar_decrypt(C.caesar_encrypt(pt, 11), 11) == pt
    r["Monoalphabetic"] = C.mono_decrypt(C.mono_encrypt(pt, key), key) == pt
    r["Vigenere"] = C.vigenere_decrypt(C.vigenere_encrypt(pt, "MEDICAL"), "MEDICAL") == pt
    r["Columnar"] = C.columnar_decrypt(C.columnar_encrypt(pt, perm), perm, n) == pt
    cc, pd = C.hill_encrypt(pt, K); r["Hill (mod 36)"] = C.hill_decrypt(cc, K, pd) == pt
    r["Mono + columnar"] = C.mono_decrypt(C.columnar_decrypt(C.columnar_encrypt(C.mono_encrypt(pt,
key), perm), perm, n), key) == pt
    sizes[n] = r
W["sizes"] = sizes
json.dump(W, open("worked.json", "w"), indent=1)
json.dump({"original": RK.ATTACKS, "after": RK.AFTER, "impact_scale": RK.IMPACT_SCALE,
```

```
"original_ranked": [r[0] for r in RK.ranked(RK.ATTACKS)], "after_ranked": [r[0] for r in RK.ranked(RK.AFTER)]}, open("risk.json", "w"), indent=1)
print(W["caesar"]["cipher"], W["mono"]["cipher"], W["vig"]["cipher"], W["col"]["cipher"], W["hill"]["cipher"], sep="\n")
print(W["col"]["grid"]); print(sizes[100000]); print(json.load(open("risk.json"))["original_ranked"], json.load(open("risk.json"))["after_ranked"])
```

Appendix F – test_all.py

test_all.py

```
"""test_all.py - correctness and boundary tests.  Usage:  python test_all.py   (prints expected vs actual, exit code 1 on failure)"""
import sys, random
import numpy as np
import classical as C
import imgsec as I
import blp
import notes
import attacks as A

rows = []
def check(name, expected, actual):
    ok = expected == actual
    rows.append((name, str(expected)[:34], str(actual)[:34], "PASS" if ok else "FAIL"))
def raises(name, f):
    try:
        f(); actual = "no error"
    except ValueError:
        actual = "ValueError"
    check(name, "ValueError", actual)

K = [[1, 1, 3], [1, 2, 1], [2, 8, 1]]
# --- Caesar
check("Caesar MEET AT NOON, k=4", "QIIXEXRSSR", C.caesar_encrypt("MEET AT NOON", 4))
check("Caesar round trip, k=26 (identity)", "HELLO", C.caesar_decrypt(C.caesar_encrypt("Hello", 26), 26))
raises("Caesar key must be an integer", lambda: C.caesar_encrypt("A", 3.5))
check("Caesar empty input", "", C.caesar_encrypt("", 5))
# --- monoalphabetic
key = C.keyword_key("HEALTHCARE")
check("Mono keyword alphabet", "HEALTCRBDFGIJKMNOPQSUVWXYZ", key)
```

```
check("Mono digits/punctuation removed", "DOSAGEMG", C.mono_decrypt(C.mono_encrypt("Dosage 500 mg!", key), key))
raises("Mono key with repeated letters", lambda: C.mono_encrypt("A", "A" * 26))
# --- Vigenere
check("Vigenere ATTACKATDAWN / LEMON (textbook)", "LXFOPVEFRNHR", C.vigenere_encrypt("ATTACKATDAWN", "LEMON"))
check("Vigenere repeated letters", "AAAAAAAAAA", C.vigenere_decrypt(C.vigenere_encrypt("AAAAAAAAAA", "KEY"), "KEY"))
raises("Vigenere empty key", lambda: C.vigenere_encrypt("HELLO", "123"))
check("Vigenere-36 AMOX 500MG round trip", "AMOX500MG", C.vigenere_decrypt(C.vigenere_encrypt("AMOX 500MG", "KEY7", C.A36), "KEY7", C.A36))
# --- transposition
perm = C.keyword_perm("ZEBRAS")
check("Columnar ZEBRAS textbook example", "EVLNXACDTXESEAXROFOXDEECXWIREE", C.columnar_encrypt("WE ARE DISCOVERED FLEE AT ONCE", perm))
check("Columnar round trip with padding", "WEAREDISCOVEREDFLEEATONCE", C.columnar_decrypt(C.columnar_encrypt("WEAREDISCOVEREDFLEEATONCE", perm), perm, 25))
raises("Columnar key not a permutation", lambda: C.columnar_encrypt("ABC", [0, 0, 1]))
# --- Hill
c, p = C.hill_encrypt("PT-20481", K)
check("Hill PT-20481 encrypts to", "KJ4OMCLYJ", c)
check("Hill PT-20481 round trip (7 symbols, pad 2)", "PT20481", C.hill_decrypt(c, K, p))
check("Hill non-multiple length (10 symbols)", "AMOX500MG", C.hill_decrypt(*[C.hill_encrypt("AMOX 500MG", K)[0], K, C.hill_encrypt("AMOX 500MG", K)[1]]))
check("Hill empty input", "", C.hill_encrypt("", K)[0])
check("Hill K * K^-1 = identity", [[1, 0, 0], [0, 1, 0], [0, 0, 1]], C.mat_mul(K, C.mat_inv(K, 36), 36))
check("Hill det(K) mod 36 and inverse of det", (7, 31), (C.det(K, 36), pow(C.det(K, 36), -1, 36)))
raises("Hill key invertible mod 26 but not mod 36", lambda: C.hill_encrypt("ABC", [[6, 24, 1], [13, 16, 10], [20, 17, 15]]))
pairs = [("PAT", C.hill_encrypt("PAT", K)[0]), ("IEN", C.hill_encrypt("IEN", K)[0]), ("PTD", C.hill_encrypt("PTD", K)[0])]
check("Hill known-plaintext attack recovers K", K, C.hill_kpa(pairs, 3)[0])
```

```
check("Number of invertible 3x3 matrices mod 2", 168, C.count_invertible(3, 2))
# --- large messages, all schemes
big = notes.corpus(1, 100000)[:100000]
check("100,000 letters: Caesar/mono/Vigenere/columnar", [True] * 4, [
    C.caesar_decrypt(C.caesar_encrypt(big, 5), 5) == big, C.mono_decrypt(C.mono_encrypt(big, key), key) ==
big,
    C.vigenere_decrypt(C.vigenere_encrypt(big, "MEDICAL"), "MEDICAL") == big,
    C.columnar_decrypt(C.columnar_encrypt(big, [4, 2, 6, 0, 3, 5, 1]), [4, 2, 6, 0, 3, 5, 1], len(big)) == big])
hc, hp = C.hill_encrypt(big, K)
check("100,000 letters: Hill (mod 36)", True, C.hill_decrypt(hc, K, hp) == big)
# --- cryptanalysis sanity
pt = big[:600]
check("Caesar brute force finds k=11", 11, A.caesar_break(C.caesar_encrypt(pt, 11))[0])
check("Vigenere break finds the key", "MEDICAL", A.vigenere_break(C.vigenere_encrypt(big[:1400],
"MEDICAL"))[0])
# --- images and steganography
img = I.phantom()
check("Image shift decrypt = original", True, bool(np.array_equal(I.shift_decrypt(I.shift_encrypt(img, 77), 77),
img)))
check("Image permutation decrypt = original", True, bool(np.array_equal(I.perm_decrypt(I.perm_encrypt(img,
5), 5), img)))
check("LSB embed/extract", "PT-20481 ALLERGY", I.lsb_extract(I.lsb_embed(img, "PT-20481
ALLERGY")))
check("LSB embed/extract (key-shuffled)", "RX AMOX 500MG", I.lsb_extract(I.lsb_embed(img, "RX AMOX
500MG", key=3), key=3))
check("LSB empty message", "", I.lsb_extract(I.lsb_embed(img, "")))
check("LSB changes each pixel by at most 1", True, int(np.abs(I.lsb_embed(img, "X" * 500).astype(int) -
img.astype(int)).max()) <= 1)
raises("LSB message larger than capacity", lambda: I.lsb_embed(img, "A" * 9000))
circ = " ".join(["word"] * 100)
check("Text stego round trip", "HI", I.text_extract(I.text_embed(circ, "HI")))
raises("Text stego circular too short", lambda: I.text_embed("a b c", "HELLO"))
```

```
# --- Bell-LaPadula
check("BLP nurse reads Secret oncology: deny", "DENY", blp.decide("Nurse Meena", "read", "oncology diagnosis")[0])
check("BLP specialist same level, other compartment", "DENY", blp.decide("Dr Iyer (psychiatrist)", "read", "oncology diagnosis")[0])
check("BLP doctor writes down to billing: deny", "DENY", blp.decide("Dr Rao (oncologist)", "write", "billing file")[0])
check("BLP trusted declassification", "ALLOW (trusted)", blp.decide("Medical Director", "write", "lab summary (declassified)", "declassify")[0])

w = max(len(r[0]) for r in rows)
print(f"{'Test':{w}}  {'Expected':34}  {'Actual':34}  Result")
for r in rows:
    print(f"{r[0]:{w}}  {r[1]:34}  {r[2]:34}  {r[3]}")
fails = sum(1 for r in rows if r[3] == "FAIL")
print(f"\n{len(rows) - fails} of {len(rows)} tests passed")
sys.exit(1 if fails else 0)
```

Appendix G – Program Output

python test_all.py

Test	Expected	Actual	Result
Caesar MEET AT NOON, k=4	QIIXEXRSSR	QIIXEXRSSR	PASS
Caesar round trip, k=26 (identity)	HELLO	HELLO	PASS
Caesar key must be an integer	ValueError	ValueError	PASS
Caesar empty input			PASS
Mono keyword alphabet	HEALTCRBDFGIJKMNOPQSUVWXYZ	HEALTCRBDFGIJKMNOPQSUVWXYZ	PASS
Mono digits/punctuation removed	DOSAGEMG	DOSAGEMG	PASS
Mono key with repeated letters	ValueError	ValueError	PASS
Vigenere ATTACKATDAWN / LEMON (textbook)	LXFOPVEFRNHR	LXFOPVEFRNHR	PASS
Vigenere repeated letters	AAAAAAAAAA	AAAAAAAAAA	PASS
Vigenere empty key	ValueError	ValueError	PASS
Vigenere-36 AMOX 500MG round trip	AMOX500MG	AMOX500MG	PASS
Columnar ZEBRAS textbook example	EVLNXACDTXESEAXROFOXDEECXWIREE	EVLNXACDTXESEAXROFOXDEECXWIREE	PASS
Columnar round trip with padding	WEAREDISCOVEREDFLEEATONCE	WEAREDISCOVEREDFLEEATONCE	PASS
Columnar key not a permutation	ValueError	ValueError	PASS
Hill PT-20481 encrypts to	KJ4OMCLYJ	KJ4OMCLYJ	PASS
Hill PT-20481 round trip (7 symbols, pad 2)	PT20481	PT20481	PASS
Hill non-multiple length (10 symbols)	AMOX500MG	AMOX500MG	PASS
Hill empty input			PASS
Hill K * K^-1 = identity	[[1, 0, 0], [0, 1, 0], [0, 0, 1]]	[[1, 0, 0], [0, 1, 0], [0, 0, 1]]	PASS
Hill det(K) mod 36 and inverse of det	(7, 31)	(7, 31)	PASS
Hill key invertible mod 26 but not mod 36	ValueError	ValueError	PASS
Hill known-plaintext attack recovers K	[[1, 1, 3], [1, 2, 1], [2, 8, 1]]	[[1, 1, 3], [1, 2, 1], [2, 8, 1]]	PASS
Number of invertible 3x3 matrices mod 2	168	168	PASS

```
100,000 letters: Caesar/mono/Vigenere/columnar  [True, True, True, True]  [True, True, True, True]  PASS
100,000 letters: Hill (mod 36)                  True                      True                      PASS
Caesar brute force finds k=11                   11                        11                        PASS
Vigenere break finds the key                    MEDICAL                   MEDICAL                   PASS
Image shift decrypt = original                  True                      True                      PASS
Image permutation decrypt = original            True                      True                      PASS
LSB embed/extract                               PT-20481 ALLERGY          PT-20481 ALLERGY          PASS
LSB embed/extract (key-shuffled)                RX AMOX 500MG             RX AMOX 500MG             PASS
LSB empty message                                                                                   PASS
LSB changes each pixel by at most 1             True                      True                      PASS
LSB message larger than capacity                ValueError                ValueError                PASS
Text stego round trip                           HI                        HI                        PASS
Text stego circular too short                   ValueError                ValueError                PASS
BLP nurse reads Secret oncology: deny           DENY                      DENY                      PASS
BLP specialist same level, other compartment    DENY                      DENY                      PASS
BLP doctor writes down to billing: deny         DENY                      DENY                      PASS
BLP trusted declassification                    ALLOW (trusted)           ALLOW (trusted)           PASS

40 of 40 tests passed
```

python run_all.py (log of the experiments; seed 2026)

```
plaintext IC (100000 letters): 0.0733
keyspace Caesar log2 = 4.6
keyspace Monoalphabetic log2 = 88.4
keyspace Vigenere m=7 log2 = 32.9
keyspace Vigenere m=12 log2 = 56.4
keyspace Columnar 7 cols log2 = 12.3
keyspace Columnar 10 cols log2 = 21.8
keyspace Columnar 18 cols log2 = 52.5
keyspace Hill 3x3 mod 26 log2 = 40.6
keyspace Hill 3x3 mod 36 log2 = 44.1
```

keyspace Mono + columnar(7) log2 = 100.7
keyspace One-time pad, 336 letters log2 = 1579.3
round trip 200 {'caesar': True, 'mono': True, 'vig': True, 'col': True, 'hill': True, 'product': True}
round trip 1000 {'caesar': True, 'mono': True, 'vig': True, 'col': True, 'hill': True, 'product': True}
round trip 10000 {'caesar': True, 'mono': True, 'vig': True, 'col': True, 'hill': True, 'product': True}
IC 10k {'plaintext': 0.0719, 'caesar': 0.0719, 'mono': 0.0719, 'vig': 0.042, 'col': 0.0718, 'hill': 0.0288, 'product': 0.0719, 'random26': 0.0385, 'ra
ndom36': 0.0278}
worked breaks {'caesar': 1.0, 'mono': 1.0, 'vig': 1.0, 'col': 1.0} kasiski [(7, 118), (2, 73), (14, 62), (3, 48), (21, 39)] key MEDICAL
length 56 {'caesar': 1.0, 'mono': 0.3, 'monofreq': 0.0, 'vig': 0.0, 'col': 0.86, 'product': 0.0} 3 s
length 112 {'caesar': 1.0, 'mono': 0.57, 'monofreq': 0.0, 'vig': 0.33, 'col': 0.99, 'product': 0.08} 6 s
length 224 {'caesar': 1.0, 'mono': 0.87, 'monofreq': 0.0, 'vig': 0.7, 'col': 1.0, 'product': 0.25} 10 s
length 448 {'caesar': 1.0, 'mono': 0.97, 'monofreq': 0.0, 'vig': 1.0, 'col': 1.0, 'product': 0.67} 15 s
length 896 {'caesar': 1.0, 'mono': 1.0, 'monofreq': 0.0, 'vig': 1.0, 'col': 1.0, 'product': 0.83} 20 s
length 1792 {'caesar': 1.0, 'mono': 1.0, 'monofreq': 0.0, 'vig': 1.0, 'col': 1.0, 'product': 1.0} 25 s
length 3584 {'caesar': 1.0, 'mono': 1.0, 'monofreq': 0.1, 'vig': 1.0, 'col': 1.0, 'product': 1.0} 33 s
length 7168 {'caesar': 1.0, 'mono': 1.0, 'monofreq': 0.07, 'vig': 1.0, 'col': 1.0, 'product': 1.0} 43 s
mono attack on product (n=1792) mean accuracy 0.071
hill KPA match True P(3 random blocks invertible) 0.195 blocks needed mean 4.916
timing 100000 enc ms {'Caesar': 12.17, 'Monoalphabetic': 3.01, 'Vigenere': 10.94, 'Columnar': 2.66, 'Hill (mod 36)': 50.79, 'Mono + columnar': 6.79}
error propagation {'Caesar': 1.0, 'Monoalphabetic': 1.0, 'Vigenere': 1.0, 'Columnar': 0.99, 'Hill (mod 36)': 2.89}
malleability {'plain': 'AMOX500MG', 'key': 'KEY7', 'cipher': 'KQCUF4OJQ', 'forged': 'KQCUJ4OJQ', 'decrypted_forged': 'AMOX900MG'}
lab table {'none': {'cell change': 0.0, 'row swap': 0.0, 'cell swap in a row': 0.0}, 'sum': {'cell change': 0.975, 'row swap': 0.0, 'cell swap in a ro
w': 0.0}, 'weighted': {'cell change': 0.963, 'row swap': 0.943, 'cell swap in a row': 1.0}}
insider forgery accepted: {'row_before': 'R06POTASS0339MMOLX', 'row_after': 'R06POTASS9999MMOLX', 'accepted': True}
image {'orig': {'entropy': 5.188, 'corr': 0.95, 'edge': 5.328}, 'shift': {'entropy': 5.188, 'corr': 0.966, 'edge': 4.464}, 'perm': {'entropy': 5.188,

'corr': 0.003, 'edge': 68.267}, 'both': {'entropy': 5.188, 'corr': 0.008, 'edge': 57.83}, 'ok_shift': True, 'ok_perm': True, 'ok_both': True}
stego {'bits': 632, 'capacity_bits': 65536, 'psnr_seq': 71.82402260149391, 'psnr_key': 71.27133171507177, 'max_diff': 1, 'changed_pixels': 280, 'extra
ct_ok': True, 'cnr_before': 0.3596305686213711, 'cnr_after_100pct': 0.3586975480581606} {'quality': [95, 90, 75, 50], 'ber': [0.49841772151898733, 0.5
205696202531646, 0.5363924050632911, 0.370253164556962], 'recovered': [False, False, False, False]}
text stego {'words': 654, 'capacity_bits': 637, 'capacity_chars': 79, 'message': 'PT20481 ALLERGY PENICILLIN', 'extract_ok': True, 'double_spaces': 77
, 'after_whitespace_collapse': '', 'after_retype_single_space': ''}
depth {'N': [1, 2, 5, 10, 15, 20, 30, 50, 100, 300], 'same_key': [0.13640873015873017, 0.10788690476190477, 0.08263888888888889, 0.190922619047619, 0.
839781746031746, 0.886904761904762, 0.9037698412698413, 0.9092261904761904, 0.9186507936507936, 0.9231150793650794], 'otp': 0.04204695767195768, 'head
er_kpa': {'cipher_first20': 'WOLXMGTTFPKHPOYCWLID', 'header': 'PATIENTIDPTDIAGNOSIS', 'keystream12': 'HOSPITALCARE', 'true_key': 'HOSPITALCARE', 'matc
h': True}}

BLP 1 Dr Rao (oncologist) read oncology diagnosis -> ALLOW

BLP 2 Nurse Meena read oncology diagnosis -> DENY

BLP 3 Nurse Meena read lab values -> ALLOW

BLP 4 Billing clerk read prescription -> DENY

BLP 5 Dr Iyer (psychiatrist) read oncology diagnosis -> DENY

BLP 6 Dr Rao (oncologist) read HIV test result -> DENY

BLP 7 Medical Director read psychiatric notes -> ALLOW

BLP 8 Lab tech Arun write lab values -> ALLOW

BLP 9 Lab tech Arun write billing file -> DENY

BLP 10 Billing clerk write lab values -> ALLOW

BLP 11 Dr Rao (oncologist) write billing file -> DENY

BLP 12 Medical Director write lab summary (declassified) -> ALLOW (trusted)

BLP 13 Nurse Meena read allergy record -> ALLOW (override)

BLP 14 Patient read health leaflet -> ALLOW

BLP 15 Patient read prescription -> DENY

BLP 16 Dr Rao (oncologist) write executive summary -> ALLOW